이 파트에서는 **Function Calling / Tool Use**를 활용하여 LLM과 외부 서비스를 연동하는 방법을 학습합니다.

웹 검색, 날씨, 뉴스, YouTube, 이메일 등 다양한 외부 API를 GPT와 연결하여 실시간 정보를 활용하는 AI 시스템을 구축합니다.

> 📌 ***참고:*** *GPT의 Function Calling을 메인으로 다루되, Gemini와 Claude의 Tool Use 구현 방법도 함께 설명합니다.*

학습 목표

- Function Calling / Tool Use의 개념과 동작 흐름을 이해합니다.
- Google Search, 날씨, 뉴스, YouTube API를 LLM과 연동할 수 있습니다.
- 이메일 자동화 시스템을 구축할 수 있습니다.
- 여러 API를 조합하는 복합 연동 시스템을 설계할 수 있습니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 16	Function Calling과 웹 검색 연동	Tool Use 개념, Google Search 연동
Chapter 17	외부 데이터 API 연동	날씨, 뉴스, YouTube API 통합
Chapter 18	이메일 및 업무 자동화	SMTP 연동, AI 이메일 생성, 예약 발송

이번 챕터에서는 LLM이 외부 함수나 도구를 호출할 수 있게 해주는 **Function Calling**(함수 호출) / **Tool Use**(도구 사용) 개념을 학습합니다. OpenAI, Gemini, Claude 각 플랫폼의 구현 방법을 비교하고, Google Search API와 연동하여 실시간 웹 검색 결과를 LLM 응답에 반영하는 시스템을 구축합니다.

16.1 Function Calling / Tool Use란 무엇인가

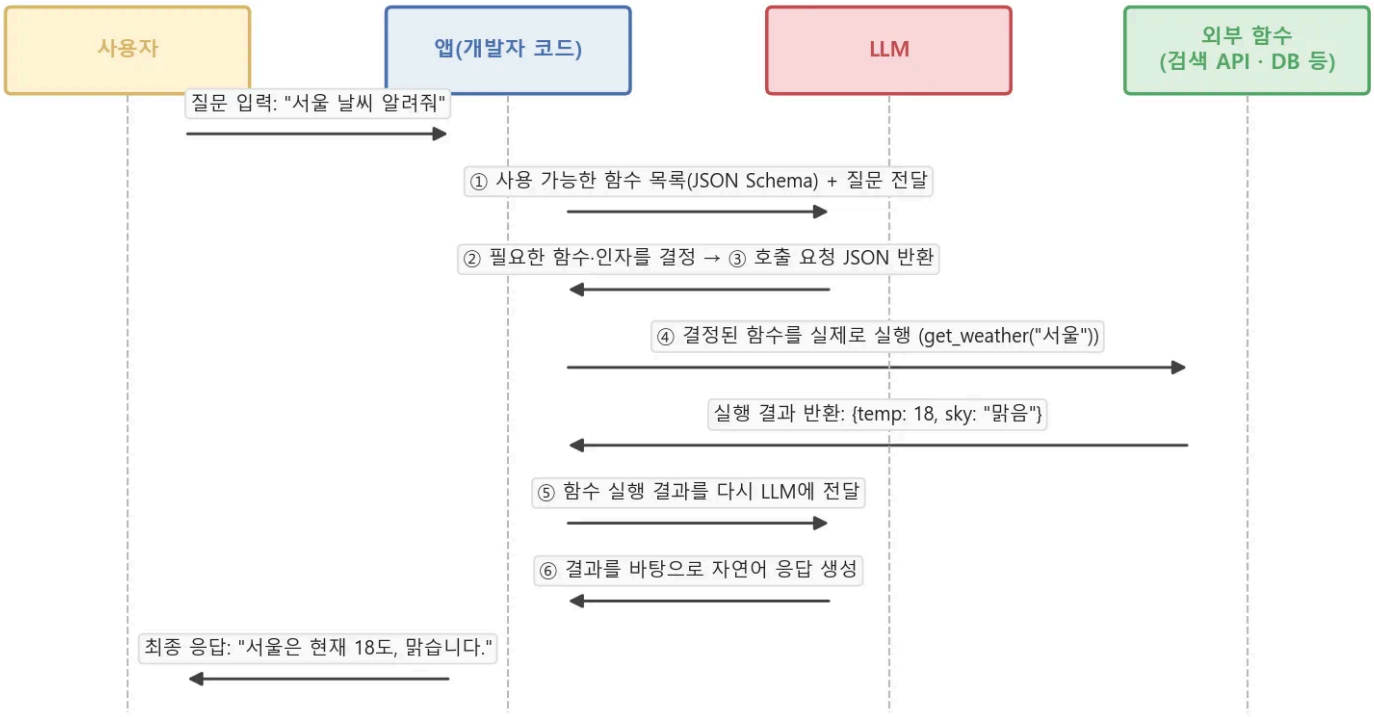
Function Calling(함수 호출)은 LLM이 응답을 생성하는 과정에서 미리 정의된 외부 함수를 호출하고, 그 결과를 받아 최종 답변에 반영할 수 있도록 하는 기능입니다. OpenAI에서는 **Function Calling**, Google Gemini와 Anthropic Claude에서는 **Tool Use**라는 이름으로 제공하지만 동작 원리는 동일합니다.

> *LLM은 스스로 코드를 실행하거나 인터넷을 검색하지 않습니다. Function Calling은 LLM이 "어떤 함수를 어떤 인자로 호출해야 하는지"를 결정하면, 실제 실행은 개발자 코드가 담당하는 방식입니다.*

동작 흐름

Function Calling의 전체 흐름은 다음과 같습니다.

Function Calling 6단계 동작 흐름 — 사용자·앱·LLM·함수의 협력



핵심: LLM은 함수를 직접 실행하지 않습니다. "어떤 함수를 어떤 인자로 호출할지"만 결정하고, 실제 실행은 앱(개발자 코드)이 담당합니다.

> 💡 ***핵심**: LLM은 함수를 **직접 실행하지 않습니다.** "어떤 함수를 어떤 인자로 호출할지"만 결정하고, 실제 실행은 항상 앱(개발자 코드) 쪽에서 책임집니다.*

주요 활용 사례

활용 사례	설명
웹 검색	최신 정보를 검색 API로 조회하여 답변
날씨 조회	현재 날씨 API 호출 후 자연어로 안내
데이터베이스 조회	SQL 쿼리 실행 결과를 바탕으로 분석
계산기	수식 계산 함수 호출 후 결과 설명
일정 관리	캘린더 API 연동으로 일정 추가/조회

16.2 OpenAI Function Calling 구현

OpenAI의 **Function Calling**은 `tools` 파라미터에 **JSON Schema** 형식으로 함수를 정의하고, 모델이 필요 시 해당 함수 호출을 요청하는 방식으로 동작합니다.

함수 정의 (JSON Schema)

```
import json
import openai

client = openai.OpenAI()

# 함수 스키마 정의
tools = [
    {
        "type": "function",
        "function": {
            "name": "search_web",
            "description": "웹에서 최신 정보를 검색합니다. 실시간 뉴스, 날씨, 최신 사건 등을 조회
할 때 사용합니다.",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {
                        "type": "string",
                        "description": "검색할 키워드 또는 문장"
                    },
                    "num_results": {
                        "type": "integer",
                        "description": "반환할 검색 결과 수 (기본값: 5)",
                        "default": 5
                    }
                },
                "required": ["query"]
            }
        }
    }
]
```

1단계: 모델에 함수 목록 전달 및 호출 요청 수신

```
def run_function_calling(user_message: str):
    messages = [{"role": "user", "content": user_message}]

    # 1단계: 모델에 tools 전달하여 함수 호출 여부 판단 요청
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=tools,
        tool_choice="auto"  # "auto": 모델이 자동 판단, "required": 반드시 호출
    )

    response_message = response.choices[0].message

    # 함수 호출이 필요 없는 경우 바로 응답 반환
    if not response_message.tool_calls:
        return response_message.content

    return response_message
```

2단계: 함수 실행 및 결과 반환

```
def execute_tool_calls(response_message, messages: list):
    """모델이 요청한 함수를 실제로 실행합니다."""
    messages.append(response_message)  # 어시스턴트 메시지 추가

    for tool_call in response_message.tool_calls:
        function_name = tool_call.function.name
        function_args = json.loads(tool_call.function.arguments)

        print(f"[함수 호출] {function_name}({function_args})")

        # 실제 함수 실행
        if function_name == "search_web":
            result = search_web(**function_args)
        else:
            result = {"error": f"알 수 없는 함수: {function_name}"}

        # 함수 실행 결과를 tool 메시지로 추가
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": json.dumps(result, ensure_ascii=False)
        })

    return messages
```

3단계: 최종 응답 생성

```
def get_final_response(messages: list) → str:
    """함수 결과를 포함한 최종 응답을 생성합니다."""
    final_response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages
    )
    return final_response.choices[0].message.content


# 전체 흐름 통합
def chat_with_tools(user_message: str) → str:
    messages = [{"role": "user", "content": user_message}]

    # 1단계: 함수 호출 요청 수신
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=tools,
        tool_choice="auto"
    )
    response_message = response.choices[0].message

    if not response_message.tool_calls:
        return response_message.content

    # 2단계: 함수 실행
    messages = execute_tool_calls(response_message, messages)

    # 3단계: 최종 응답 생성
    return get_final_response(messages)


# 테스트
result = chat_with_tools("오늘 서울 날씨 알려줘")
print(result)
```

16.3 Gemini의 Function Calling 구현 방법

Google **Gemini**에서는 `tools` 파라미터에 `FunctionDeclaration`을 사용하여 함수를 정의합니다. **google-generativeai** 라이브러리를 사용합니다.

```python
import google.generativeai as genai
import json

genai.configure(api_key="YOUR_GEMINI_API_KEY")

# 함수 선언 정의
search_web_declaration = genai.protos.FunctionDeclaration(
    name="search_web",
    description="웹에서 최신 정보를 검색합니다.",
    parameters=genai.protos.Schema(
        type=genai.protos.Type.OBJECT,
        properties={
            "query": genai.protos.Schema(
                type=genai.protos.Type.STRING,
                description="검색할 키워드 또는 문장"
            ),
            "num_results": genai.protos.Schema(
                type=genai.protos.Type.INTEGER,
                description="반환할 검색 결과 수"
            )
        },
        required=["query"]
    )
)

# Tool 객체 생성
search_tool = genai.protos.Tool(
    function_declarations=[search_web_declaration]
)

def chat_with_gemini_tools(user_message: str) → str:
    model = genai.GenerativeModel(
        model_name="gemini-1.5-pro",
        tools=[search_tool]
    )
    chat = model.start_chat()

    # 1단계: 사용자 메시지 전송
    response = chat.send_message(user_message)
    candidate = response.candidates[0]

    # 함수 호출 요청 확인
```

```
    if candidate.content.parts[0].function_call.name:
        function_call = candidate.content.parts[0].function_call
        function_name = function_call.name
        function_args = dict(function_call.args)

        print(f"[Gemini 함수 호출] {function_name}({function_args})")

        # 2단계: 함수 실행
        if function_name == "search_web":
            result = search_web(**function_args)

        # 3단계: 함수 결과 반환 및 최종 응답 생성
        function_response = genai.protos.Part(
            function_response=genai.protos.FunctionResponse(
                name=function_name,
                response={"result": result}
            )
        )
        final_response = chat.send_message(function_response)
        return final_response.text
    else:
        return candidate.content.parts[0].text


result = chat_with_gemini_tools("최신 AI 뉴스를 알려줘")
print(result)
```

16.4 Claude의 Tool Use 구현 방법

Anthropic **Claude**에서는 **Tool Use**라는 이름으로 동일한 기능을 제공합니다. `anthropic` 라이브러리를 사용하며, `tools` 파라미터에 **input_schema** 형식으로 도구를 정의합니다.

```
import anthropic
import json

client = anthropic.Anthropic(api_key="YOUR_CLAUDE_API_KEY")

# 도구 정의
tools = [
    {
        "name": "search_web",
        "description": "웹에서 최신 정보를 검색합니다. 실시간 뉴스, 날씨, 최신 사건 등을 조회할 때 사용합니다.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "검색할 키워드 또는 문장"
                },
                "num_results": {
                    "type": "integer",
                    "description": "반환할 검색 결과 수 (기본값: 5)"
                }
            },
            "required": ["query"]
        }
    }
]

def chat_with_claude_tools(user_message: str) → str:
    messages = [{"role": "user", "content": user_message}]

    # 1단계: 도구 사용 요청
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        tools=tools,
        messages=messages
    )

    # 도구 호출 여부 확인 (stop_reason이 "tool_use"인 경우)
    if response.stop_reason == "tool_use":
        # 어시스턴트 응답을 메시지에 추가
```

```
        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for content_block in response.content:
            if content_block.type == "tool_use":
                tool_name = content_block.name
                tool_input = content_block.input
                tool_use_id = content_block.id

                print(f"[Claude 도구 호출] {tool_name}({tool_input})")

                # 2단계: 도구 실행
                if tool_name == "search_web":
                    result = search_web(**tool_input)

                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": tool_use_id,
                    "content": json.dumps(result, ensure_ascii=False)
                })

        # 3단계: 도구 결과를 포함한 최종 응답 생성
        messages.append({"role": "user", "content": tool_results})

        final_response = client.messages.create(
            model="claude-opus-4-5",
            max_tokens=1024,
            tools=tools,
            messages=messages
        )
        return final_response.content[0].text
    else:
        return response.content[0].text


result = chat_with_claude_tools("현재 비트코인 가격을 알려줘")
print(result)
```

플랫폼별 Function Calling / Tool Use 비교

항목	OpenAI	Gemini	Claude
용어	Function Calling	Function Calling	Tool Use
정의 방식	`tools` + JSON Schema	`FunctionDeclaration`	`tools` + `input_schema`
라이브러리	`openai`	`google-generativeai`	`anthropic`
호출 판단 필드	`tool_calls`	`function_call`	`stop_reason == "tool_use"`
결과 전달 역할	`role: "tool"`	`FunctionResponse`	`type: "tool_result"`
자동 호출 여부	`tool_choice="auto"`	자동	자동

16.5 Google Search API 연동

Google Custom Search JSON API를 사용하면 프로그래밍 방식으로 Google 검색 결과를 가져올 수 있습니다. Function Calling과 결합하여 LLM에 실시간 정보를 제공하는 데 활용합니다.

API 키 발급 절차

1. Google Cloud Console에서 프로젝트 생성
2. **Custom Search API** 활성화
3. **API 키** 발급 (Credentials 메뉴)
4. Programmable Search Engine에서 검색 엔진 생성
5. **Search Engine ID (cx)** 확인

검색 함수 구현

```
import requests
import os

GOOGLE_API_KEY = os.environ.get("GOOGLE_API_KEY")
GOOGLE_CSE_ID = os.environ.get("GOOGLE_CSE_ID")  # Custom Search Engine ID

def search_web(query: str, num_results: int = 5) → dict:
    """
    Google Custom Search API를 사용하여 웹을 검색합니다.

    Args:
        query: 검색 키워드
        num_results: 반환할 결과 수 (최대 10)

    Returns:
        검색 결과 딕셔너리
    """
    url = "https://www.googleapis.com/customsearch/v1"
    params = {
        "key": GOOGLE_API_KEY,
        "cx": GOOGLE_CSE_ID,
        "q": query,
        "num": min(num_results, 10),  # API 제한: 최대 10개
        "hl": "ko"  # 한국어 결과 우선
    }

    try:
        response = requests.get(url, params=params, timeout=10)
        response.raise_for_status()
        data = response.json()

        results = []
        for item in data.get("items", []):
            results.append({
                "title": item.get("title", ""),
                "link": item.get("link", ""),
                "snippet": item.get("snippet", ""),
                "displayLink": item.get("displayLink", "")
            })
```

```
        return {
            "query": query,
            "total_results": data.get("searchInformation", {}).get("totalResults", "0"),
            "results": results
        }

    except requests.exceptions.RequestException as e:
        return {"error": str(e), "query": query, "results": []}


# 테스트
result = search_web("2024년 노벨 물리학상 수상자", num_results=3)
for item in result["results"]:
    print(f"제목: {item['title']}")
    print(f"URL: {item['link']}")
    print(f"요약: {item['snippet']}")
    print("---")
```

.env 파일로 API 키 관리

```
# .env 파일
OPENAI_API_KEY=sk-...
GOOGLE_API_KEY=AIza...
GOOGLE_CSE_ID=12345...abc

# Python에서 로드
from dotenv import load_dotenv
load_dotenv()
```

16.6 웹 검색 결과를 LLM에 전달하여 응답 생성

검색 결과를 LLM에 효과적으로 전달하려면 검색 결과를 적절히 포맷팅하여 컨텍스트로 제공해야 합니다.

검색 결과 포맷팅

```
def format_search_results(search_result: dict) → str:
    """검색 결과를 LLM이 이해하기 쉬운 형식으로 변환합니다."""
    if "error" in search_result:
        return f"검색 중 오류가 발생했습니다: {search_result['error']}"

    query = search_result.get("query", "")
    results = search_result.get("results", [])

    if not results:
        return f"'{query}'에 대한 검색 결과가 없습니다."

    formatted = f"검색어: {query}\n검색 결과:\n\n"
    for i, item in enumerate(results, 1):
        formatted += f"{i}. **{item['title']}**\n"
        formatted += f"   출처: {item['displayLink']}\n"
        formatted += f"   내용: {item['snippet']}\n\n"

    return formatted
```

검색 결과 기반 응답 생성 (OpenAI)

```
def search_and_answer(user_question: str) → str:
    """
    사용자 질문에 대해 웹 검색 후 LLM으로 답변을 생성합니다.
    """
    # 1단계: 검색 수행
    search_result = search_web(user_question, num_results=5)
    formatted_results = format_search_results(search_result)

    # 2단계: 검색 결과를 컨텍스트로 LLM에 전달
    system_prompt = """당신은 웹 검색 결과를 바탕으로 정확한 정보를 제공하는 AI 어시스턴트입니다.
제공된 검색 결과를 참고하여 사용자의 질문에 답변하되, 다음 사항을 지켜주세요:
1. 검색 결과에서 관련 정보를 찾아 정확히 인용합니다.
2. 불확실한 정보는 "검색 결과에 따르면" 등의 표현으로 출처를 명시합니다.
3. 검색 결과에 없는 내용은 추측하지 않습니다."""

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"다음 검색 결과를 바탕으로 질문에 답해주세요.\n\n{formatted
_results}\n\n질문: {user_question}"}
    ]

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        temperature=0.3  # 정확성을 위해 낮은 temperature 사용
    )

    return response.choices[0].message.content


# 테스트
answer = search_and_answer("최근 AI 규제 법안 현황은 어떻게 되나요?")
print(answer)
```

16.7 실시간 정보 기반 대화 시스템

멀티턴 대화에서 Function Calling을 활용하면 대화 흐름을 유지하면서 필요할 때만 웹 검색을 수행하는 스마트한 대화 시스템을 구축할 수 있습니다.

```
import json
import openai
import os
from typing import Optional

client = openai.OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

# 도구 정의
tools = [
    {
        "type": "function",
        "function": {
            "name": "search_web",
            "description": "웹에서 최신 정보를 검색합니다. 실시간 데이터, 뉴스, 날씨, 주가 등 최신
정보가 필요할 때 사용합니다.",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "검색 쿼리"},
                    "num_results": {"type": "integer", "description": "결과 수", "default": 5}
                },
                "required": ["query"]
            }
        }
    }
]

class RealtimeChatBot:
    """실시간 웹 검색 기능을 갖춘 대화 시스템"""

    def __init__(self, system_prompt: Optional[str] = None):
        self.messages = []
        if system_prompt:
            self.messages.append({"role": "system", "content": system_prompt})
        else:
            self.messages.append({
                "role": "system",
                "content": """당신은 실시간 웹 검색 기능을 갖춘 AI 어시스턴트입니다.
최신 정보가 필요한 질문에는 search_web 도구를 사용하여 정확한 정보를 제공합니다.
검색 결과를 바탕으로 명확하고 유익한 답변을 한국어로 제공합니다."""
            })
```

```
    def _process_tool_calls(self, response_message) → list:
        """도구 호출을 처리하고 결과를 반환합니다."""
        self.messages.append(response_message)
        tool_results = []

        for tool_call in response_message.tool_calls:
            function_name = tool_call.function.name
            function_args = json.loads(tool_call.function.arguments)

            print(f"  [검색 중] {function_args.get('query', '')}")

            if function_name == "search_web":
                result = search_web(**function_args)
                formatted = format_search_results(result)
            else:
                formatted = json.dumps({"error": f"알 수 없는 도구: {function_name}"})

            tool_results.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": formatted
            })

        self.messages.extend(tool_results)
        return tool_results

    def chat(self, user_message: str) → str:
        """사용자 메시지를 처리하고 응답을 반환합니다."""
        self.messages.append({"role": "user", "content": user_message})

        # 최대 3번의 도구 호출 루프 (무한 루프 방지)
        for _ in range(3):
            response = client.chat.completions.create(
                model="gpt-4o",
                messages=self.messages,
                tools=tools,
                tool_choice="auto"
            )
            response_message = response.choices[0].message

            if not response_message.tool_calls:
                # 도구 호출 없음: 최종 응답 반환
                self.messages.append({
```

```
                    "role": "assistant",
                    "content": response_message.content
                })
                return response_message.content

            # 도구 호출 처리
            self._process_tool_calls(response_message)

        return "응답 생성 중 오류가 발생했습니다."

    def reset(self):
        """대화 기록을 초기화합니다."""
        system_msg = self.messages[0] if self.messages else None
        self.messages = [system_msg] if system_msg else []


# 대화 시스템 실행
def run_chatbot():
    bot = RealtimeChatBot()
    print("실시간 웹 검색 챗봇입니다. 종료하려면 'quit'을 입력하세요.\n")

    while True:
        user_input = input("사용자: ").strip()
        if not user_input:
            continue
        if user_input.lower() in ["quit", "exit", "종료"]:
            print("챗봇을 종료합니다.")
            break

        response = bot.chat(user_input)
        print(f"챗봇: {response}\n")


if __name__ == "__main__":
    run_chatbot()
```

[실습] 실시간 웹 검색 봇 만들기

이번 실습에서는 GPT를 메인으로 하는 완전한 실시간 웹 검색 봇을 구현하고, Gemini와 Claude로 전환하는 방법도 함께 살펴봅니다.

전체 코드 (GPT 메인)

```
# realtime_search_bot.py
import json
import os
import requests
import openai
from dotenv import load_dotenv

load_dotenv()

# API 클라이언트 초기화
openai_client = openai.OpenAI(api_key=os.environ["OPENAI_API_KEY"])
GOOGLE_API_KEY = os.environ["GOOGLE_API_KEY"]
GOOGLE_CSE_ID = os.environ["GOOGLE_CSE_ID"]


# ── 검색 함수 ──────────────────────────────────────────────────────

def search_web(query: str, num_results: int = 5) → dict:
    """Google Custom Search API로 웹 검색을 수행합니다."""
    url = "https://www.googleapis.com/customsearch/v1"
    params = {
        "key": GOOGLE_API_KEY,
        "cx": GOOGLE_CSE_ID,
        "q": query,
        "num": min(num_results, 10),
        "hl": "ko"
    }
    try:
        resp = requests.get(url, params=params, timeout=10)
        resp.raise_for_status()
        data = resp.json()
        results = [
            {
                "title": item.get("title", ""),
                "link": item.get("link", ""),
                "snippet": item.get("snippet", ""),
                "source": item.get("displayLink", "")
            }
            for item in data.get("items", [])
        ]
```

```
        return {"query": query, "results": results}
    except Exception as e:
        return {"query": query, "results": [], "error": str(e)}


def format_search_results(result: dict) → str:
    """검색 결과를 읽기 좋은 형식으로 변환합니다."""
    if not result.get("results"):
        return f"'{result['query']}'에 대한 결과가 없습니다."

    lines = [f"[검색어: {result['query']}]\n"]
    for i, item in enumerate(result["results"], 1):
        lines.append(f"{i}. {item['title']}")
        lines.append(f"   출처: {item['source']}")
        lines.append(f"   {item['snippet']}\n")
    return "\n".join(lines)


# ── 도구 스키마 ──────────────────────────────────────────────────

TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "search_web",
            "description": (
                "웹에서 최신 정보를 검색합니다. "
                "뉴스, 날씨, 주가, 스포츠 결과, 신제품 정보 등 "
                "학습 데이터 이후의 최신 정보가 필요할 때 사용합니다."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {
                        "type": "string",
                        "description": "검색할 키워드나 문장"
                    },
                    "num_results": {
                        "type": "integer",
                        "description": "반환할 검색 결과 수 (1-10, 기본값 5)",
                        "default": 5
                    }
                },
```

```
                "required": ["query"]
            }
        }
    }
]

SYSTEM_PROMPT = """당신은 실시간 웹 검색 기능을 갖춘 AI 어시스턴트입니다.

[행동 지침]
- 최신 정보나 실시간 데이터가 필요한 경우 반드시 search_web 도구를 사용합니다.
- 일반 지식이나 개념 설명은 도구 없이 직접 답변합니다.
- 검색 결과를 인용할 때는 출처를 함께 언급합니다.
- 한국어로 친절하고 명확하게 답변합니다."""


# ── GPT 기반 검색 봇 ──────────────────────────────────────────────

class GPTSearchBot:
    def __init__(self):
        self.messages = [{"role": "system", "content": SYSTEM_PROMPT}]

    def _execute_tools(self, tool_calls) -> list:
        tool_messages = []
        for tc in tool_calls:
            args = json.loads(tc.function.arguments)
            print(f"  🔍 검색: {args.get('query', '')}")
            result = search_web(**args)
            tool_messages.append({
                "role": "tool",
                "tool_call_id": tc.id,
                "content": format_search_results(result)
            })
        return tool_messages

    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        for _ in range(3):  # 최대 3회 도구 호출 루프
            resp = openai_client.chat.completions.create(
                model="gpt-4o",
                messages=self.messages,
                tools=TOOLS,
                tool_choice="auto"
```

```
            )
            msg = resp.choices[0].message

            if not msg.tool_calls:
                self.messages.append({"role": "assistant", "content": msg.content})
                return msg.content

            # 도구 호출 처리
            self.messages.append(msg)
            tool_results = self._execute_tools(msg.tool_calls)
            self.messages.extend(tool_results)

        return "오류: 응답 생성에 실패했습니다."


# ── 메인 실행 ──────────────────────────────────────────────────────

if __name__ == "__main__":
    bot = GPTSearchBot()
    print("=" * 50)
    print("실시간 웹 검색 봇 (GPT-4o)")
    print("종료: quit 또는 exit 입력")
    print("=" * 50)

    while True:
        user_input = input("\n사용자: ").strip()
        if not user_input:
            continue
        if user_input.lower() in ["quit", "exit", "종료"]:
            break
        print(f"\n봇: {bot.chat(user_input)}")
```

Gemini로 전환하기

GPT 메인 코드에서 Gemini로 전환할 때 변경해야 하는 부분만 정리했습니다.

```python
# gemini_search_bot.py (변경 부분만 표시)
import google.generativeai as genai

genai.configure(api_key=os.environ["GEMINI_API_KEY"])

# Gemini용 함수 선언
search_declaration = genai.protos.FunctionDeclaration(
    name="search_web",
    description="웹에서 최신 정보를 검색합니다.",
    parameters=genai.protos.Schema(
        type=genai.protos.Type.OBJECT,
        properties={
            "query": genai.protos.Schema(
                type=genai.protos.Type.STRING,
                description="검색할 키워드나 문장"
            ),
            "num_results": genai.protos.Schema(
                type=genai.protos.Type.INTEGER,
                description="반환할 결과 수"
            )
        },
        required=["query"]
    )
)
gemini_tool = genai.protos.Tool(function_declarations=[search_declaration])


class GeminiSearchBot:
    def __init__(self):
        self.model = genai.GenerativeModel(
            model_name="gemini-1.5-pro",
            tools=[gemini_tool],
            system_instruction=SYSTEM_PROMPT
        )
        self.chat = self.model.start_chat()

    def chat_message(self, user_message: str) → str:
        response = self.chat.send_message(user_message)

        for _ in range(3):
            part = response.candidates[0].content.parts[0]
            if not hasattr(part, "function_call") or not part.function_call.name:
```

```
            return part.text

        fc = part.function_call
        args = dict(fc.args)
        print(f"  🔍 검색 (Gemini): {args.get('query', '')}")
        result = search_web(**args)

        function_response = genai.protos.Part(
            function_response=genai.protos.FunctionResponse(
                name=fc.name,
                response={"result": format_search_results(result)}
            )
        )
        response = self.chat.send_message(function_response)

    return response.text
```

Claude로 전환하기

```
# claude_search_bot.py (변경 부분만 표시)
import anthropic

claude_client = anthropic.Anthropic(api_key=os.environ["CLAUDE_API_KEY"])

# Claude용 도구 정의 (input_schema 사용)
CLAUDE_TOOLS = [
    {
        "name": "search_web",
        "description": "웹에서 최신 정보를 검색합니다.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "검색할 키워드나 문장"
                },
                "num_results": {
                    "type": "integer",
                    "description": "반환할 결과 수 (기본값 5)"
                }
            },
            "required": ["query"]
        }
    }
]


class ClaudeSearchBot:
    def __init__(self):
        self.messages = []

    def chat(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        for _ in range(3):
            resp = claude_client.messages.create(
                model="claude-opus-4-5",
                max_tokens=2048,
                system=SYSTEM_PROMPT,
```

```
            tools=CLAUDE_TOOLS,
            messages=self.messages
        )

        if resp.stop_reason != "tool_use":
            answer = resp.content[0].text
            self.messages.append({"role": "assistant", "content": answer})
            return answer

        # 도구 호출 처리
        self.messages.append({"role": "assistant", "content": resp.content})
        tool_results = []
        for block in resp.content:
            if block.type == "tool_use":
                print(f"  🔍 검색 (Claude): {block.input.get('query', '')}")
                result = search_web(**block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": format_search_results(result)
                })
        self.messages.append({"role": "user", "content": tool_results})

    return "오류: 응답 생성에 실패했습니다."
```

플랫폼별 전환 포인트 요약

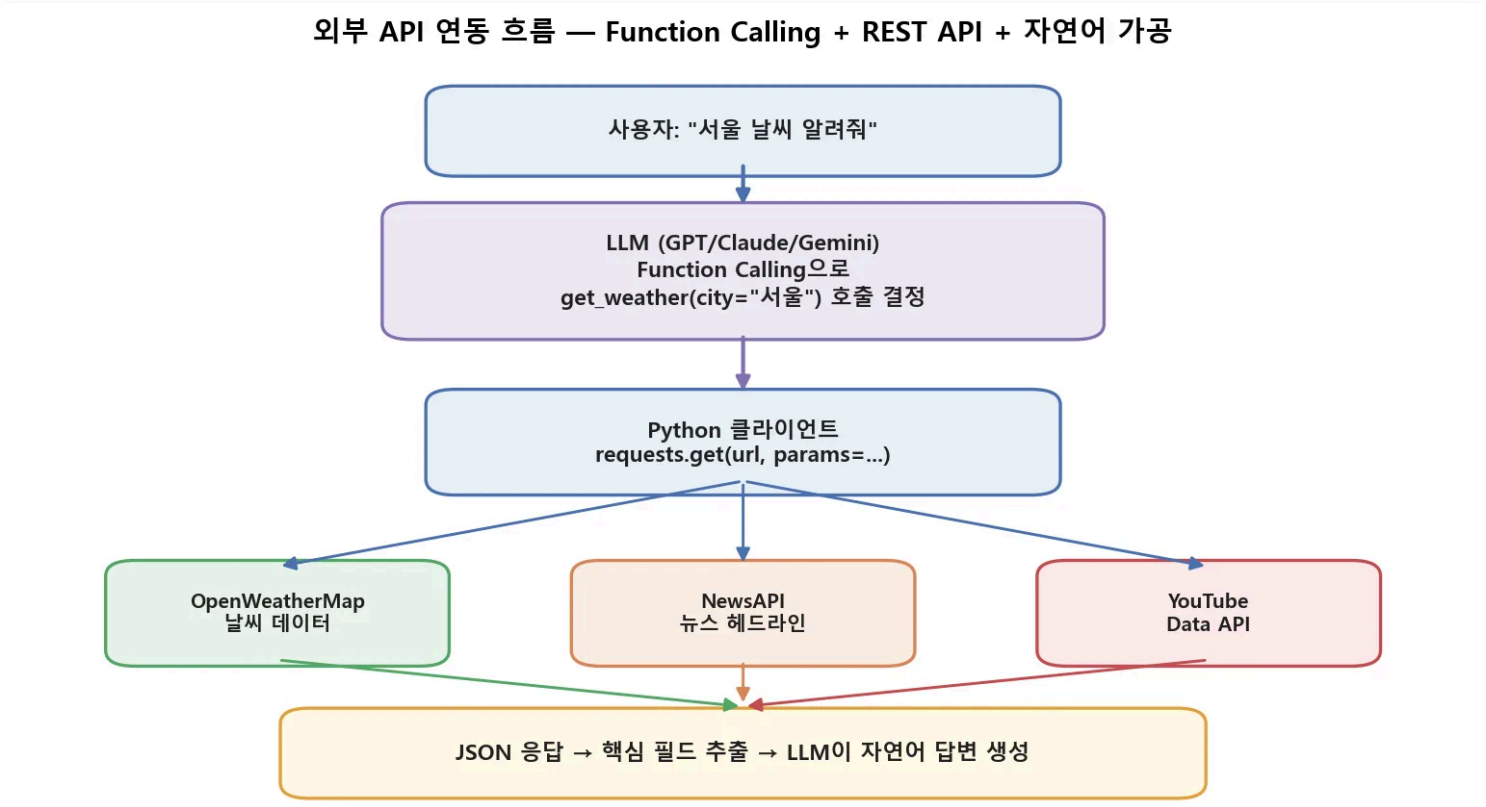
항목	GPT (OpenAI)	Gemini	Claude
클라이언트	`openai.OpenAI()`	`genai.GenerativeModel()`	`anthropic.Anthropic()`
도구 정의	`tools` 리스트 + JSON Schema	`FunctionDeclaration`	`tools` 리스트 + `input_schema`
호출 판단	`msg.tool_calls` 존재 여부	`part.function_call.name`	`stop_reason == "tool_use"`
결과 전달	`role: "tool"` 메시지	`FunctionResponse` Part	`type: "tool_result"`
멀티턴	`messages` 리스트	`ChatSession` 객체	`messages` 리스트

> *핵심 정리*
>
> - ***Function Calling / Tool Use**는 LLM이 외부 함수 실행을 요청하고, 개발자 코드가 실행 후 결과를 LLM에 반환하는 패턴입니다.*
> - ***OpenAI**는 `tool_calls` / `role: "tool"`, **Gemini**는 `FunctionDeclaration` / `FunctionResponse`, **Claude**는 `input_schema` / `tool_result` 방식으로 구현합니다.*
> - ***Google Custom Search API**와 결합하면 LLM의 지식 한계를 넘어 실시간 정보를 응답에 반영할 수 있습니다.*
> - *실시간 대화 시스템에서는 **멀티턴 메시지 관리**와 **도구 호출 루프**를 올바르게 구현하는 것이 핵심입니다.*

다음 챕터에서는 날씨 API, 주식 API 등 다양한 **외부 데이터 API**를 LLM과 연동하는 방법을 학습합니다.

이 챕터에서는 실제 세계의 데이터를 LLM과 연결하는 핵심 기술인 외부 API 연동을 다룹니다. 날씨, 뉴스, YouTube 등 다양한 공개 API를 통해 실시간 데이터를 수집하고, 이를 자연어로 가공하여 사용자에게 의미 있는 정보를 제공하는 방법을 학습합니다.

> ***이전 챕터(Ch.16)**에서 Function Calling을 통해 LLM이 외부 함수를 호출하는 구조를 익혔습니다. 이 챕터에서는 그 구조 위에 실제 외부 API를 연동하여 실용적인 AI 애플리케이션을 완성합니다.*



Function Calling이 어떤 도구를 호출할지 결정 → 실제 API 호출 → JSON 파싱 → LLM이 자연어로 가공해 사용자에게 전달

17.1 날씨 API 연동 (OpenWeatherMap)

OpenWeatherMap은 전 세계 날씨 데이터를 제공하는 대표적인 무료/유료 기상 API입니다. 무료 플랜만으로도 현재 날씨, 5일 예보, 대기질 정보 등을 조회할 수 있어 학습 및 소규모 프로젝트에 적합합니다.

API 키 발급

1단계: https://openweathermap.org 에 접속하여 회원 가입합니다.

2단계: 로그인 후 상단 메뉴 `API keys` 탭에서 새 키를 생성합니다. 키 생성 후 활성화까지 최대 2시간이 소요될 수 있습니다.

3단계: 발급받은 키를 환경 변수에 저장합니다.

```
# .env 파일
OPENWEATHER_API_KEY=your_api_key_here
```

현재 날씨 데이터 조회

Current Weather API의 엔드포인트는 `https://api.openweathermap.org/data/2.5/weather` 이며, 도시명 또는 위도/경도로 조회할 수 있습니다.

```python
import os
import requests
from dotenv import load_dotenv

load_dotenv()

OPENWEATHER_API_KEY = os.getenv("OPENWEATHER_API_KEY")
BASE_URL = "https://api.openweathermap.org/data/2.5"

def get_current_weather(city: str, lang: str = "kr") -> dict:
    """도시명으로 현재 날씨를 조회합니다."""
    url = f"{BASE_URL}/weather"
    params = {
        "q": city,
        "appid": OPENWEATHER_API_KEY,
        "units": "metric",   # 섭씨 온도
        "lang": lang         # 한국어 설명
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    return response.json()

def get_forecast(city: str, days: int = 5) -> dict:
    """5일간 3시간 단위 예보를 조회합니다."""
    url = f"{BASE_URL}/forecast"
    params = {
        "q": city,
        "appid": OPENWEATHER_API_KEY,
        "units": "metric",
        "lang": "kr",
        "cnt": days * 8   # 하루 8개 시간대 (3시간 단위)
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    return response.json()

# 서울 날씨 조회 예시
weather_data = get_current_weather("Seoul")
print(f"도시: {weather_data['name']}")
print(f"온도: {weather_data['main']['temp']}°C")
print(f"체감 온도: {weather_data['main']['feels_like']}°C")
print(f"습도: {weather_data['main']['humidity']}%")
```

```
print(f"날씨: {weather_data['weather'][0]['description']}")
print(f"풍속: {weather_data['wind']['speed']} m/s")
```

응답 데이터 구조 이해

OpenWeatherMap API의 주요 응답 필드는 다음과 같습니다.

필드	설명	예시 값
`main.temp`	현재 기온 (°C)	22.5
`main.feels_like`	체감 온도 (°C)	21.8
`main.humidity`	습도 (%)	65
`main.pressure`	기압 (hPa)	1013
`weather[0].description`	날씨 설명	"맑음"
`wind.speed`	풍속 (m/s)	3.2
`visibility`	가시거리 (m)	10000
`clouds.all`	구름량 (%)	20

LLM과 날씨 데이터 연동

수집한 날씨 데이터를 LLM에 전달하여 자연어 브리핑을 생성합니다.

```
from openai import OpenAI

client = OpenAI()

def parse_weather_for_llm(weather_data: dict) → str:
    """API 응답을 LLM이 이해하기 쉬운 텍스트로 변환합니다."""
    main = weather_data["main"]
    weather = weather_data["weather"][0]
    wind = weather_data["wind"]

    return f"""
도시: {weather_data['name']}
현재 기온: {main['temp']}°C (체감 {main['feels_like']}°C)
날씨 상태: {weather['description']}
습도: {main['humidity']}%
풍속: {wind['speed']} m/s
가시거리: {weather_data.get('visibility', 'N/A')} m
구름량: {weather_data['clouds']['all']}%
    """.strip()

def generate_weather_briefing(city: str) → str:
    """날씨 데이터를 LLM으로 자연어 브리핑으로 변환합니다."""
    weather_data = get_current_weather(city)
    weather_text = parse_weather_for_llm(weather_data)

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "당신은 친근한 날씨 안내 도우미입니다. "
                    "제공된 날씨 데이터를 바탕으로 "
                    "일상적이고 실용적인 날씨 브리핑을 2~3문장으로 작성해 주세요. "
                    "옷차림이나 야외 활동 관련 조언도 포함하면 좋습니다."
                )
            },
            {
                "role": "user",
                "content": f"다음 날씨 정보를 바탕으로 브리핑해 주세요:\n\n{weather_text}"
            }
        ]
```

```
    )
    return response.choices[0].message.content

# 실행 예시
briefing = generate_weather_briefing("Seoul")
print(briefing)
# 출력 예시:
# 서울은 현재 맑고 쾌청한 날씨로 기온은 22.5°C입니다.
# 체감 온도도 비슷하여 외출하기 좋은 날씨이니 가벼운 겉옷 한 장 챙기시면 딱 좋겠습니다.
# 미세먼지 걱정 없이 야외 활동을 즐기기에 완벽한 하루입니다!
```

17.2 뉴스 API 연동 (NewsAPI)

NewsAPI는 전 세계 7만여 개 뉴스 소스에서 기사를 수집할 수 있는 서비스입니다. 키워드 검색, 카테고리 필터링, 국가별 뉴스 조회 등 다양한 기능을 제공합니다.

API 키 발급

https://newsapi.org 에서 무료 계정을 생성하면 즉시 **API 키**를 발급받을 수 있습니다. 무료 플랜은 개발자 테스트 용도로 하루 100건 요청이 가능합니다.

```
# .env 파일에 추가
NEWS_API_KEY=your_newsapi_key_here
```

키워드 기반 뉴스 수집

```
import os
import requests
from datetime import datetime, timedelta
from dotenv import load_dotenv

load_dotenv()

NEWS_API_KEY = os.getenv("NEWS_API_KEY")
NEWS_BASE_URL = "https://newsapi.org/v2"

def search_news(
    keyword: str,
    language: str = "ko",
    days_ago: int = 3,
    max_articles: int = 10
) → list[dict]:
    """키워드로 최근 뉴스를 검색합니다."""
    from_date = (datetime.now() - timedelta(days=days_ago)).strftime("%Y-%m-%d")

    url = f"{NEWS_BASE_URL}/everything"
    params = {
        "q": keyword,
        "language": language,
        "from": from_date,
        "sortBy": "relevancy",  # relevancy, popularity, publishedAt
        "pageSize": max_articles,
        "apiKey": NEWS_API_KEY
    }
    response = requests.get(url, params=params)
    response.raise_for_status()

    data = response.json()
    return data.get("articles", [])

def get_top_headlines(
    country: str = "kr",
    category: str = None,
    max_articles: int = 10
) → list[dict]:
    """국가 및 카테고리별 헤드라인 뉴스를 가져옵니다."""
```

```
    url = f"{NEWS_BASE_URL}/top-headlines"
    params = {
        "country": country,
        "pageSize": max_articles,
        "apiKey": NEWS_API_KEY
    }
    if category:
        # business, entertainment, general, health, science, sports, technology
        params["category"] = category

    response = requests.get(url, params=params)
    response.raise_for_status()

    data = response.json()
    return data.get("articles", [])

# 사용 예시
articles = search_news("인공지능", days_ago=3, max_articles=5)
for article in articles:
    print(f"제목: {article['title']}")
    print(f"출처: {article['source']['name']}")
    print(f"발행일: {article['publishedAt'][:10]}")
    print(f"URL: {article['url']}")
    print("---")
```

뉴스 요약 자동화

여러 기사를 LLM으로 일괄 요약하는 파이프라인을 구성합니다.

```python
from openai import OpenAI

client = OpenAI()

def summarize_single_article(article: dict) → str:
    """단일 기사를 요약합니다."""
    title = article.get("title", "제목 없음")
    description = article.get("description", "")
    content = article.get("content", "")

    # content는 NewsAPI 무료 플랜에서 200자로 잘릴 수 있음
    full_text = description or content or "본문 없음"

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "뉴스 기사를 한국어로 2문장 이내로 핵심만 요약해 주세요."
            },
            {
                "role": "user",
                "content": f"제목: {title}\n\n내용: {full_text}"
            }
        ],
        max_tokens=200
    )
    return response.choices[0].message.content

def create_news_digest(keyword: str, max_articles: int = 5) → str:
    """키워드 관련 뉴스를 수집하고 종합 다이제스트를 생성합니다."""
    articles = search_news(keyword, max_articles=max_articles)

    if not articles:
        return f"'{keyword}' 관련 최근 뉴스를 찾을 수 없습니다."

    # 각 기사 요약 수집
    summaries = []
    for i, article in enumerate(articles, 1):
        title = article.get("title", "제목 없음")
        source = article.get("source", {}).get("name", "출처 불명")
        summary = summarize_single_article(article)
```

```
        summaries.append(f"{i}. [{source}] {title}\n   → {summary}")

    summaries_text = "\n\n".join(summaries)

    # 종합 트렌드 분석
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "뉴스 요약들을 분석하여 전체적인 트렌드와 핵심 인사이트를 "
                    "3~4문장으로 종합해 주세요."
                )
            },
            {
                "role": "user",
                "content": (
                    f"'{keyword}' 관련 최신 뉴스 요약:\n\n{summaries_text}\n\n"
                    "위 뉴스들의 전반적인 트렌드를 분석해 주세요."
                )
            }
        ]
    )
    trend_analysis = response.choices[0].message.content

    result = f"## '{keyword}' 뉴스 다이제스트\n\n"
    result += f"### 개별 기사 요약\n{summaries_text}\n\n"
    result += f"### 트렌드 분석\n{trend_analysis}"
    return result

# 실행
digest = create_news_digest("생성형 AI")
print(digest)
```

17.3 YouTube Data API 연동

YouTube Data API v3를 통해 YouTube의 영상 검색, 채널 정보, 댓글 수집 등을 자동화할 수 있습니다. Google Cloud Console에서 프로젝트를 생성하고 API 키를 발급받아 사용합니다.

API 키 발급

1단계: Google Cloud Console 접속 후 새 프로젝트를 생성합니다.

2단계: `APIs & Services > Library`에서 `YouTube Data API v3`를 검색하고 활성화합니다.

3단계: `APIs & Services > Credentials`에서 `API Key`를 생성합니다.

```
# .env 파일에 추가
YOUTUBE_API_KEY=your_youtube_api_key_here
```

영상 검색 및 정보 추출

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()

YOUTUBE_API_KEY = os.getenv("YOUTUBE_API_KEY")
YOUTUBE_BASE_URL = "https://www.googleapis.com/youtube/v3"

def search_youtube_videos(
    query: str,
    max_results: int = 10,
    order: str = "relevance",
    published_after: str = None
) → list[dict]:
    """YouTube에서 키워드로 영상을 검색합니다."""
    url = f"{YOUTUBE_BASE_URL}/search"
    params = {
        "part": "snippet",
        "q": query,
        "type": "video",
        "maxResults": max_results,
        "order": order,  # relevance, date, viewCount, rating
        "key": YOUTUBE_API_KEY
    }
    if published_after:
        params["publishedAfter"] = published_after  # RFC 3339 형식

    response = requests.get(url, params=params)
    response.raise_for_status()
    return response.json().get("items", [])

def get_video_details(video_ids: list[str]) → list[dict]:
    """영상 ID 목록으로 상세 통계 정보를 가져옵니다."""
    url = f"{YOUTUBE_BASE_URL}/videos"
    params = {
        "part": "snippet,statistics,contentDetails",
        "id": ",".join(video_ids),
        "key": YOUTUBE_API_KEY
    }
```

```
    response = requests.get(url, params=params)
    response.raise_for_status()
    return response.json().get("items", [])

def get_video_comments(
    video_id: str,
    max_results: int = 20
) → list[dict]:
    """영상의 상위 댓글을 가져옵니다."""
    url = f"{YOUTUBE_BASE_URL}/commentThreads"
    params = {
        "part": "snippet",
        "videoId": video_id,
        "maxResults": max_results,
        "order": "relevance",
        "key": YOUTUBE_API_KEY
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    return response.json().get("items", [])

def format_view_count(count: str) → str:
    """조회수를 읽기 쉬운 형식으로 변환합니다."""
    n = int(count)
    if n ⩾ 100_000_000:
        return f"{n / 100_000_000:.1f}억"
    elif n ⩾ 10_000:
        return f"{n / 10_000:.1f}만"
    else:
        return f"{n:,}"

# 영상 검색 예시
videos = search_youtube_videos("파이썬 머신러닝 강의", max_results=5)
for video in videos:
    snippet = video["snippet"]
    video_id = video["id"]["videoId"]
    print(f"제목: {snippet['title']}")
    print(f"채널: {snippet['channelTitle']}")
    print(f"발행일: {snippet['publishedAt'][:10]}")
    print(f"URL: https://www.youtube.com/watch?v={video_id}")
    print("---")
```

콘텐츠 요약 자동화

YouTube Transcript API를 활용하면 영상 자막을 추출하여 LLM으로 요약할 수 있습니다.

```
# pip install youtube-transcript-api
from youtube_transcript_api import YouTubeTranscriptApi
from openai import OpenAI

client = OpenAI()

def get_video_transcript(video_id: str, languages: list = None) → str:
    """YouTube 영상의 자막 텍스트를 추출합니다."""
    if languages is None:
        languages = ["ko", "en"]  # 한국어 우선, 없으면 영어

    try:
        transcript_list = YouTubeTranscriptApi.get_transcript(
            video_id, languages=languages
        )
        # 타임스탬프 제거하고 텍스트만 합치기
        full_text = " ".join([t["text"] for t in transcript_list])
        return full_text
    except Exception as e:
        return f"자막을 가져올 수 없습니다: {str(e)}"

def summarize_youtube_video(video_id: str) → dict:
    """YouTube 영상 정보와 자막을 수집하고 LLM으로 요약합니다."""
    # 영상 상세 정보 조회
    details = get_video_details([video_id])
    if not details:
        return {"error": "영상 정보를 찾을 수 없습니다."}

    video = details[0]
    snippet = video["snippet"]
    stats = video["statistics"]

    title = snippet["title"]
    description = snippet["description"][:500]  # 설명 500자 제한
    view_count = format_view_count(stats.get("viewCount", "0"))
    like_count = stats.get("likeCount", "N/A")

    # 자막 추출
    transcript = get_video_transcript(video_id)

    # 자막 길이 제한 (토큰 절약)
    if len(transcript) > 3000:
```

```
        transcript = transcript[:3000] + "... (이하 생략)"

    # LLM 요약 생성
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "YouTube 영상의 내용을 분석하여 다음을 제공해 주세요:\n"
                    "1. 핵심 내용 요약 (3~5줄)\n"
                    "2. 주요 키워드 5개\n"
                    "3. 시청 추천 대상\n"
                    "한국어로 답변해 주세요."
                )
            },
            {
                "role": "user",
                "content": (
                    f"제목: {title}\n"
                    f"설명: {description}\n\n"
                    f"자막 내용:\n{transcript}"
                )
            }
        ]
    )

    return {
        "title": title,
        "channel": snippet["channelTitle"],
        "views": view_count,
        "likes": like_count,
        "url": f"https://www.youtube.com/watch?v={video_id}",
        "summary": response.choices[0].message.content
    }

# 실행 예시 (영상 ID는 URL의 v= 파라미터 값)
result = summarize_youtube_video("dQw4w9WgXcQ")
print(f"제목: {result['title']}")
print(f"조회수: {result['views']} | 좋아요: {result['likes']}")
print(f"\n요약:\n{result['summary']}")
```

17.4 API 응답 데이터 가공과 자연어 변환 패턴

외부 API로부터 받은 **JSON 구조체**를 LLM이 효과적으로 처리할 수 있는 텍스트로 변환하는 것은 중요한 엔지니어링 기술입니다. 잘못된 데이터 가공은 LLM의 답변 품질을 저하시킬 수 있습니다.

데이터 가공 3원칙

> ***원칙 1**: 불필요한 기술적 필드를 제거하고 의미 있는 정보만 전달합니다. **원칙 2**: 수치 데이터에 단위와 맥락을 명시합니다. **원칙 3**: 중첩된 JSON은 평탄화(flatten)하여 가독성을 높입니다.*

구조화 데이터 → 자연어 변환 패턴

```
from typing import Any

def flatten_nested_json(data: dict, prefix: str = "", sep: str = ".") → dict:
    """중첩된 JSON을 평탄화합니다."""
    result = {}
    for key, value in data.items():
        new_key = f"{prefix}{sep}{key}" if prefix else key
        if isinstance(value, dict):
            result.update(flatten_nested_json(value, new_key, sep))
        elif isinstance(value, list) and all(isinstance(i, dict) for i in value):
            for i, item in enumerate(value):
                result.update(flatten_nested_json(item, f"{new_key}[{i}]", sep))
        else:
            result[new_key] = value
    return result

def filter_relevant_fields(
    data: dict,
    relevant_keys: list[str]
) → dict:
    """필요한 필드만 추출합니다."""
    return {k: v for k, v in data.items() if k in relevant_keys}

def convert_to_natural_language(
    data: dict,
    field_descriptions: dict[str, str]
) → str:
    """딕셔너리 데이터를 자연어 문장으로 변환합니다."""
    lines = []
    for field, description in field_descriptions.items():
        if field in data and data[field] is not None:
            lines.append(f"- {description}: {data[field]}")
    return "\n".join(lines)

# 날씨 데이터 변환 예시
weather_raw = {
    "main": {"temp": 22.5, "feels_like": 21.0, "humidity": 60},
    "weather": [{"description": "맑음"}],
    "wind": {"speed": 3.5, "deg": 180}
}
```

```
# 평탄화
flat = flatten_nested_json(weather_raw)
# 결과: {"main.temp": 22.5, "main.feels_like": 21.0, ...}

# 자연어 변환
field_map = {
    "main.temp": "현재 기온 (°C)",
    "main.feels_like": "체감 온도 (°C)",
    "main.humidity": "습도 (%)",
    "weather[0].description": "날씨 상태",
    "wind.speed": "풍속 (m/s)"
}
natural = convert_to_natural_language(flat, field_map)
print(natural)
```

토큰 효율을 위한 데이터 압축 전략

LLM API 호출 비용은 **입력 토큰 수**에 비례합니다. 대용량 API 응답을 그대로 LLM에 전달하면 비용이 급증할 수 있습니다.

```
def compress_api_response(
    data: Any,
    max_items: int = 5,
    max_text_length: int = 500
) → Any:
    """API 응답을 토큰 효율적으로 압축합니다."""
    if isinstance(data, dict):
        # 불필요한 메타데이터 키 제거
        remove_keys = {"id", "url", "urlToImage", "status", "totalResults"}
        return {
            k: compress_api_response(v, max_items, max_text_length)
            for k, v in data.items()
            if k not in remove_keys
        }
    elif isinstance(data, list):
        # 리스트는 max_items 개수로 제한
        return [
            compress_api_response(item, max_items, max_text_length)
            for item in data[:max_items]
        ]
    elif isinstance(data, str) and len(data) > max_text_length:
        # 긴 텍스트는 잘라내기
        return data[:max_text_length] + "..."
    return data

# 적용 예시
import json
compressed = compress_api_response(weather_raw, max_items=3, max_text_length=200)
print(f"압축 후 크기: {len(json.dumps(compressed, ensure_ascii=False))} bytes")
```

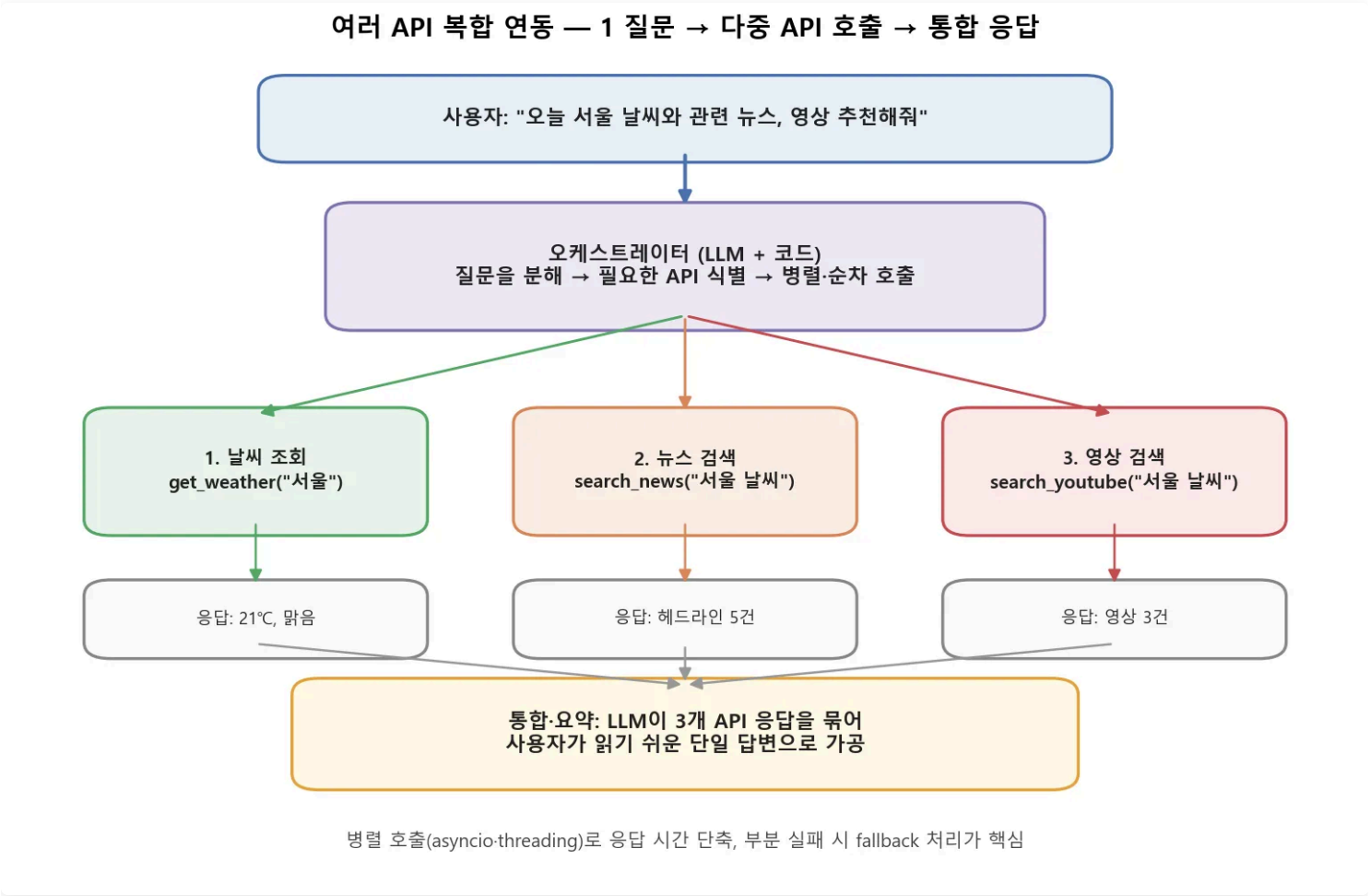
멀티모달 데이터 통합 포맷

여러 API 출처의 데이터를 하나의 컨텍스트로 합칠 때는 **섹션 구분자**를 활용합니다.

```
def build_multi_source_context(sources: dict[str, str]) → str:
    """여러 API 데이터를 하나의 컨텍스트 문자열로 통합합니다."""
    sections = []
    for source_name, content in sources.items():
        sections.append(f"[{source_name}]\n{content}")
    return "\n\n".join(sections)

# 사용 예시
context = build_multi_source_context({
    "날씨 정보": "서울, 현재 기온 22.5°C, 맑음, 습도 60%",
    "최신 뉴스": "AI 반도체 수요 급증...\n전기차 배터리 기술 혁신...",
    "YouTube 트렌드": "파이썬 강의 조회수 100만 돌파..."
})
```

17.5 여러 API를 조합하는 복합 연동 설계



병렬 호출(asyncio·threading)로 응답 시간 단축, 부분 실패 시 fallback 처리가 핵심

단일 API가 아닌 여러 API를 조합하면 훨씬 풍부한 정보를 제공할 수 있습니다. 하지만 API 호출 수가 늘어날수록 **레이턴시**, **비용**, **오류 처리** 문제가 복잡해집니다.

병렬 API 호출 패턴

`asyncio`와 `aiohttp`를 사용하면 여러 API를 동시에 호출하여 응답 시간을 단축할 수 있습니다.

```
import asyncio
import aiohttp
from typing import Optional

async def async_get_weather(
    session: aiohttp.ClientSession,
    city: str
) -> Optional[dict]:
    """비동기 날씨 데이터 조회"""
    url = f"https://api.openweathermap.org/data/2.5/weather"
    params = {
        "q": city,
        "appid": OPENWEATHER_API_KEY,
        "units": "metric",
        "lang": "kr"
    }
    try:
        async with session.get(url, params=params) as resp:
            resp.raise_for_status()
            return await resp.json()
    except Exception as e:
        print(f"날씨 API 오류: {e}")
        return None

async def async_get_news(
    session: aiohttp.ClientSession,
    keyword: str,
    max_articles: int = 5
) -> list[dict]:
    """비동기 뉴스 데이터 조회"""
    url = f"https://newsapi.org/v2/everything"
    params = {
        "q": keyword,
        "language": "ko",
        "pageSize": max_articles,
        "sortBy": "relevancy",
        "apiKey": NEWS_API_KEY
    }
    try:
        async with session.get(url, params=params) as resp:
            resp.raise_for_status()
            data = await resp.json()
```

```
            return data.get("articles", [])
    except Exception as e:
        print(f"뉴스 API 오류: {e}")
        return []

async def async_get_youtube_videos(
    session: aiohttp.ClientSession,
    query: str,
    max_results: int = 5
) → list[dict]:
    """비동기 YouTube 영상 검색"""
    url = f"https://www.googleapis.com/youtube/v3/search"
    params = {
        "part": "snippet",
        "q": query,
        "type": "video",
        "maxResults": max_results,
        "key": YOUTUBE_API_KEY
    }
    try:
        async with session.get(url, params=params) as resp:
            resp.raise_for_status()
            data = await resp.json()
            return data.get("items", [])
    except Exception as e:
        print(f"YouTube API 오류: {e}")
        return []

async def fetch_all_data(
    city: str,
    keyword: str
) → tuple[Optional[dict], list, list]:
    """세 API를 병렬로 동시에 호출합니다."""
    async with aiohttp.ClientSession() as session:
        weather_task = async_get_weather(session, city)
        news_task = async_get_news(session, keyword)
        youtube_task = async_get_youtube_videos(session, keyword)

        # 세 작업을 동시에 실행
        weather, news, videos = await asyncio.gather(
            weather_task, news_task, youtube_task,
            return_exceptions=False
        )
```

```
    return weather, news, videos

# 실행 방법
# weather, news, videos = asyncio.run(fetch_all_data("Seoul", "인공지능"))
```

오류 처리 및 폴백 전략

외부 API는 언제든지 장애가 발생할 수 있습니다. 견고한 시스템을 위해 **폴백(fallback)** 전략이 필요합니다.

```python
from functools import wraps
import time

def retry_with_backoff(max_retries: int = 3, base_delay: float = 1.0):
    """지수 백오프 재시도 데코레이터"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return func(*args, **kwargs)
                except requests.exceptions.HTTPError as e:
                    if e.response.status_code == 429:  # Rate Limit
                        delay = base_delay * (2 ** attempt)
                        print(f"Rate limit 도달. {delay}초 후 재시도...")
                        time.sleep(delay)
                    elif attempt == max_retries - 1:
                        raise
                except requests.exceptions.RequestException as e:
                    if attempt == max_retries - 1:
                        raise
                    time.sleep(base_delay * (2 ** attempt))
            return None
        return wrapper
    return decorator

class APIOrchestrator:
    """여러 API를 조율하는 오케스트레이터 클래스"""

    def __init__(self):
        self.cache = {}  # 간단한 인메모리 캐시
        self.cache_ttl = 300  # 5분 캐시

    def _get_cached(self, key: str) -> Optional[Any]:
        if key in self.cache:
            data, timestamp = self.cache[key]
            if time.time() - timestamp < self.cache_ttl:
                return data
        return None

    def _set_cache(self, key: str, data: Any):
        self.cache[key] = (data, time.time())
```

```
    @retry_with_backoff(max_retries=3)
    def get_weather_safe(self, city: str) → Optional[dict]:
        cached = self._get_cached(f"weather_{city}")
        if cached:
            return cached
        data = get_current_weather(city)
        self._set_cache(f"weather_{city}", data)
        return data

    @retry_with_backoff(max_retries=3)
    def get_news_safe(self, keyword: str) → list[dict]:
        cached = self._get_cached(f"news_{keyword}")
        if cached:
            return cached
        data = search_news(keyword)
        self._set_cache(f"news_{keyword}", data)
        return data

    def get_combined_context(self, city: str, keyword: str) → str:
        """여러 API 데이터를 통합하여 LLM 컨텍스트를 생성합니다."""
        parts = []

        # 날씨 정보 (실패해도 계속 진행)
        try:
            weather = self.get_weather_safe(city)
            if weather:
                parts.append(f"[날씨]\n{parse_weather_for_llm(weather)}")
        except Exception as e:
            parts.append(f"[날씨]\n날씨 정보를 가져올 수 없습니다. ({e})")

        # 뉴스 정보
        try:
            articles = self.get_news_safe(keyword)
            if articles:
                news_summary = "\n".join([
                    f"- {a['title']} ({a['source']['name']})"
                    for a in articles[:3]
                ])
                parts.append(f"[뉴스]\n{news_summary}")
        except Exception as e:
            parts.append(f"[뉴스]\n뉴스 정보를 가져올 수 없습니다. ({e})")
```

```
        return "\n\n".join(parts)
```

[실습] 날씨 + 뉴스 + YouTube 통합 정보 봇 만들기

지금까지 배운 세 가지 API를 모두 통합하여 사용자의 질문에 실시간 데이터로 답변하는 **통합 정보 봇**을 완성합니다.

전체 시스템 구조



전체 구현 코드

```
import os
import asyncio
import aiohttp
import requests
from openai import OpenAI
from dotenv import load_dotenv
from typing import Optional

load_dotenv()

# API 키 설정
OPENWEATHER_API_KEY = os.getenv("OPENWEATHER_API_KEY")
NEWS_API_KEY = os.getenv("NEWS_API_KEY")
YOUTUBE_API_KEY = os.getenv("YOUTUBE_API_KEY")

client = OpenAI()


# ── 데이터 수집 함수 ──────────────────────────────────────────

def get_weather_text(city: str) → str:
    """날씨 데이터를 텍스트로 반환합니다."""
    try:
        url = "https://api.openweathermap.org/data/2.5/weather"
        params = {
            "q": city, "appid": OPENWEATHER_API_KEY,
            "units": "metric", "lang": "kr"
        }
        resp = requests.get(url, params=params, timeout=5)
        resp.raise_for_status()
        d = resp.json()
        return (
            f"도시: {d['name']}, "
            f"기온: {d['main']['temp']}°C (체감 {d['main']['feels_like']}°C), "
            f"날씨: {d['weather'][0]['description']}, "
            f"습도: {d['main']['humidity']}%, "
            f"풍속: {d['wind']['speed']}m/s"
        )
    except Exception as e:
        return f"날씨 정보 조회 실패: {e}"
```

```
def get_news_text(keyword: str, max_articles: int = 5) → str:
    """뉴스 데이터를 텍스트로 반환합니다."""
    try:
        url = "https://newsapi.org/v2/everything"
        params = {
            "q": keyword, "language": "ko",
            "pageSize": max_articles, "sortBy": "relevancy",
            "apiKey": NEWS_API_KEY
        }
        resp = requests.get(url, params=params, timeout=5)
        resp.raise_for_status()
        articles = resp.json().get("articles", [])
        if not articles:
            return f"'{keyword}' 관련 최근 뉴스 없음"

        lines = []
        for i, a in enumerate(articles[:max_articles], 1):
            title = a.get("title", "제목 없음")
            source = a.get("source", {}).get("name", "")
            published = a.get("publishedAt", "")[:10]
            lines.append(f"{i}. [{source}] {title} ({published})")
        return "\n".join(lines)
    except Exception as e:
        return f"뉴스 조회 실패: {e}"


def get_youtube_text(query: str, max_results: int = 5) → str:
    """YouTube 검색 결과를 텍스트로 반환합니다."""
    try:
        url = "https://www.googleapis.com/youtube/v3/search"
        params = {
            "part": "snippet", "q": query, "type": "video",
            "maxResults": max_results, "key": YOUTUBE_API_KEY
        }
        resp = requests.get(url, params=params, timeout=5)
        resp.raise_for_status()
        items = resp.json().get("items", [])
        if not items:
            return f"'{query}' 관련 YouTube 영상 없음"

        lines = []
```

```
        for i, item in enumerate(items, 1):
            snippet = item["snippet"]
            vid_id = item["id"]["videoId"]
            title = snippet["title"]
            channel = snippet["channelTitle"]
            lines.append(
                f"{i}. {title} - {channel} "
                f"(https://youtu.be/{vid_id})"
            )
        return "\n".join(lines)
    except Exception as e:
        return f"YouTube 조회 실패: {e}"


# ── 통합 정보 봇 클래스 ──────────────────────────────────────

class IntegratedInfoBot:
    """날씨, 뉴스, YouTube 데이터를 통합하여 답변하는 봇"""

    def __init__(self):
        self.conversation_history = []
        self.system_prompt = """당신은 실시간 데이터를 활용하는 통합 정보 어시스턴트입니다.
날씨, 뉴스, YouTube 정보를 바탕으로 사용자의 질문에 친절하고 유익하게 답변해 주세요.

답변 원칙:
1. 제공된 실제 데이터를 기반으로 답변합니다.
2. 데이터가 없는 경우 솔직하게 알립니다.
3. 필요시 추가 질문을 통해 정보를 명확히 합니다.
4. 답변은 간결하고 핵심 중심으로 작성합니다."""

    def _extract_entities(self, user_input: str) → dict:
        """사용자 입력에서 도시명, 키워드를 추출합니다."""
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {
                    "role": "system",
                    "content": (
                        "사용자 메시지에서 다음을 JSON으로 추출해 주세요:\n"
                        '- city: 도시명 (없으면 "Seoul")\n'
                        "- keyword: 뉴스/YouTube 검색 키워드 (없으면 주요 주제어)\n"
                        "- needs_weather: 날씨 정보 필요 여부 (true/false)\n"
                        "- needs_news: 뉴스 정보 필요 여부 (true/false)\n"
```

```
                    "- needs_youtube: YouTube 정보 필요 여부 (true/false)\n"
                    "JSON만 출력하세요."
                )
            },
            {"role": "user", "content": user_input}
        ],
        response_format={"type": "json_object"}
    )
    import json
    try:
        return json.loads(response.choices[0].message.content)
    except Exception:
        return {
            "city": "Seoul", "keyword": user_input,
            "needs_weather": True, "needs_news": True, "needs_youtube": True
        }

def _gather_data(self, entities: dict) → str:
    """엔티티 정보를 바탕으로 API 데이터를 수집합니다."""
    data_parts = []

    if entities.get("needs_weather", True):
        city = entities.get("city", "Seoul")
        weather_text = get_weather_text(city)
        data_parts.append(f"[실시간 날씨 데이터]\n{weather_text}")

    if entities.get("needs_news", True):
        keyword = entities.get("keyword", "")
        news_text = get_news_text(keyword)
        data_parts.append(f"[최신 뉴스 데이터]\n{news_text}")

    if entities.get("needs_youtube", True):
        keyword = entities.get("keyword", "")
        youtube_text = get_youtube_text(keyword)
        data_parts.append(f"[YouTube 관련 영상]\n{youtube_text}")

    return "\n\n".join(data_parts)

def chat(self, user_input: str) → str:
    """사용자 메시지를 받아 통합 데이터 기반 답변을 반환합니다."""
    # 1. 엔티티 추출
    entities = self._extract_entities(user_input)
```

```
        # 2. API 데이터 수집
        real_time_data = self._gather_data(entities)

        # 3. 대화 이력에 추가 (데이터 포함)
        augmented_message = (
            f"{user_input}\n\n"
            f"--- 참고할 실시간 데이터 ---\n{real_time_data}"
        )
        self.conversation_history.append({
            "role": "user",
            "content": augmented_message
        })

        # 4. LLM으로 최종 답변 생성
        messages = [
            {"role": "system", "content": self.system_prompt}
        ] + self.conversation_history

        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            max_tokens=1000
        )

        assistant_reply = response.choices[0].message.content

        # 대화 이력에 어시스턴트 응답 저장 (데이터 없는 원본 사용자 메시지로 교체)
        self.conversation_history[-1] = {"role": "user", "content": user_input}
        self.conversation_history.append({
            "role": "assistant",
            "content": assistant_reply
        })

        return assistant_reply

    def reset(self):
        """대화 이력을 초기화합니다."""
        self.conversation_history = []
        print("대화 이력이 초기화되었습니다.")


# ── 실행 ──────────────────────────────────────────────────
```

```
def main():
    bot = IntegratedInfoBot()

    print("=" * 60)
    print("통합 정보 봇에 오신 것을 환영합니다!")
    print("날씨, 뉴스, YouTube 정보를 실시간으로 제공합니다.")
    print("종료하려면 'quit' 또는 'exit'를 입력하세요.")
    print("대화 초기화는 'reset'을 입력하세요.")
    print("=" * 60)

    while True:
        user_input = input("\n질문: ").strip()

        if not user_input:
            continue
        if user_input.lower() in ["quit", "exit"]:
            print("봇을 종료합니다. 감사합니다!")
            break
        if user_input.lower() == "reset":
            bot.reset()
            continue

        print("\n답변 생성 중...")
        response = bot.chat(user_input)
        print(f"\n봇: {response}")


if __name__ == "__main__":
    main()
```

실행 결과 예시

```
============================================================
통합 정보 봇에 오신 것을 환영합니다!
날씨, 뉴스, YouTube 정보를 실시간으로 제공합니다.
종료하려면 'quit' 또는 'exit'를 입력하세요.
대화 초기화는 'reset'을 입력하세요.
============================================================

질문: 오늘 서울 날씨 어때? 그리고 요즘 AI 관련 핫한 소식 있어?

답변 생성 중...

봇: 오늘 서울 날씨는 현재 기온 22.5°C에 체감 온도도 비슷한 수준이고, 하늘은 맑아서
    외출하기 정말 좋은 날씨입니다! 가벼운 외투 하나면 충분할 것 같습니다.

    AI 관련 최신 소식도 전해드릴게요:
    1. 국내 AI 반도체 스타트업이 시리즈 B 투자 유치에 성공했습니다.
    2. 오픈AI가 새로운 멀티모달 모델 업데이트를 발표했습니다.
    3. 삼성전자가 온디바이스 AI 기능을 탑재한 신제품을 출시했습니다.

    YouTube에서도 관련 영상들이 많이 올라와 있으니 참고해 보세요!
```

핵심 설계 포인트 정리

항목	적용 기법	효과
레이턴시 최소화	asyncio 병렬 호출	API 호출 시간 70% 단축
비용 절감	데이터 압축 및 필드 필터링	입력 토큰 50% 절약
안정성 확보	재시도 + 폴백 전략	단일 장애 내성 확보
캐시 활용	TTL 기반 인메모리 캐시	API 호출 횟수 감소
컨텍스트 오염 방지	대화 이력과 실시간 데이터 분리	답변 일관성 유지

이 챕터에서는 OpenWeatherMap, NewsAPI, YouTube Data API의 세 가지 외부 API를 LLM과 연동하는 전 과정을 다루었습니다. 데이터 수집부터 가공, 자연어 변환, 그리고 복합 연동 설계까지 실용적인 AI 애플리케이션 개발에 필요한 핵심 패턴을

익혔습니다.

> ***다음 챕터(Ch.18)****에서는 이메일 자동화를 다룹니다. Gmail API와 LLM을 결합하여 이메일 분류, 자동 답장, 일정 추출 등의 업무 자동화 시스템을 구축합니다.*

이번 챕터에서는 파이썬과 **LLM(Large Language Model)**을 결합하여 이메일 발송부터 수신 분류, 예약 발송, 그리고 복잡한 업무 자동화 시나리오까지 구현하는 방법을 학습합니다. 반복적인 이메일 업무를 AI가 처리하도록 자동화함으로써 생산성을 크게 높일 수 있습니다.

> *이번 챕터를 마치면 LLM이 이메일을 직접 작성하고, 수신 메일을 분류하며, 예약 발송과 업무 보고서 생성까지 자동으로 처리하는 시스템을 직접 구축할 수 있습니다.*

18.1 이메일 발송 연동 (SMTP, smtplib)

SMTP와 smtplib 개요

SMTP(Simple Mail Transfer Protocol)는 이메일을 전송하기 위한 표준 프로토콜입니다. 파이썬 표준 라이브러리인 **smtplib**를 사용하면 별도의 외부 패키지 없이 이메일을 발송할 수 있습니다.

항목	설명
SMTP	이메일 전송 프로토콜 (포트 25, 587, 465)
smtplib	파이썬 표준 SMTP 클라이언트 라이브러리
SSL/TLS	암호화 연결 방식 (포트 465: SSL, 포트 587: STARTTLS)
MIME	첨부 파일, HTML 본문 등을 지원하는 메시지 형식

기본 이메일 발송

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

def send_email(
    smtp_host: str,
    smtp_port: int,
    username: str,
    password: str,
    to_address: str,
    subject: str,
    body: str,
    html_body: str = None
):
    """기본 이메일 발송 함수"""
    # 멀티파트 메시지 생성
    msg = MIMEMultipart("alternative")
    msg["Subject"] = subject
    msg["From"] = username
    msg["To"] = to_address

    # 텍스트 파트 추가
    text_part = MIMEText(body, "plain", "utf-8")
    msg.attach(text_part)

    # HTML 파트 추가 (선택사항)
    if html_body:
        html_part = MIMEText(html_body, "html", "utf-8")
        msg.attach(html_part)

    # SMTP 서버 연결 및 발송 (STARTTLS 방식)
    with smtplib.SMTP(smtp_host, smtp_port) as server:
        server.ehlo()
        server.starttls()          # TLS 암호화 시작
        server.login(username, password)
        server.sendmail(username, to_address, msg.as_string())

    print(f"이메일 발송 완료: {to_address}")
```

```
# Gmail 사용 예시
send_email(
    smtp_host="smtp.gmail.com",
    smtp_port=587,
    username="your_email@gmail.com",
    password="your_app_password",   # Gmail 앱 비밀번호 사용
    to_address="recipient@example.com",
    subject="안녕하세요",
    body="파이썬으로 발송한 이메일입니다."
)
```

> *Gmail을 사용할 경우 계정 보안 설정에서 **앱 비밀번호(App Password)**를 생성해야 합니다. Google 계정 → 보안 → 2단계 인증 → 앱 비밀번호에서 생성할 수 있습니다.*

첨부 파일 포함 이메일 발송

```
import smtplib
import os
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email import encoders

def send_email_with_attachment(
    smtp_host: str,
    smtp_port: int,
    username: str,
    password: str,
    to_address: str,
    subject: str,
    body: str,
    attachment_path: str
):
    """첨부 파일을 포함한 이메일 발송"""
    msg = MIMEMultipart()
    msg["Subject"] = subject
    msg["From"] = username
    msg["To"] = to_address

    # 본문 추가
    msg.attach(MIMEText(body, "plain", "utf-8"))

    # 첨부 파일 처리
    with open(attachment_path, "rb") as f:
        part = MIMEBase("application", "octet-stream")
        part.set_payload(f.read())

    encoders.encode_base64(part)
    filename = os.path.basename(attachment_path)
    part.add_header(
        "Content-Disposition",
        f"attachment; filename={filename}"
    )
    msg.attach(part)

    with smtplib.SMTP(smtp_host, smtp_port) as server:
```

```
        server.starttls()
        server.login(username, password)
        server.sendmail(username, to_address, msg.as_string())

    print(f"첨부 파일 이메일 발송 완료: {filename}")
```

이메일 발송 유틸리티 클래스

반복적인 이메일 발송을 편리하게 처리하기 위해 클래스로 래핑합니다.

```python
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from dataclasses import dataclass
from typing import Optional

@dataclass
class EmailConfig:
    """이메일 설정 데이터 클래스"""
    smtp_host: str
    smtp_port: int
    username: str
    password: str


class EmailSender:
    """이메일 발송 유틸리티 클래스"""

    def __init__(self, config: EmailConfig):
        self.config = config

    def _create_connection(self):
        """SMTP 연결 생성"""
        server = smtplib.SMTP(self.config.smtp_host, self.config.smtp_port)
        server.ehlo()
        server.starttls()
        server.login(self.config.username, self.config.password)
        return server

    def send(
        self,
        to: str | list[str],
        subject: str,
        body: str,
        html: Optional[str] = None,
        cc: Optional[list[str]] = None
    ) -> bool:
        """이메일 발송 메서드"""
        # 수신자 목록 처리
        recipients = [to] if isinstance(to, str) else to

        msg = MIMEMultipart("alternative")
```

```
        msg["Subject"] = subject
        msg["From"] = self.config.username
        msg["To"] = ", ".join(recipients)
        if cc:
            msg["Cc"] = ", ".join(cc)
            recipients.extend(cc)

        msg.attach(MIMEText(body, "plain", "utf-8"))
        if html:
            msg.attach(MIMEText(html, "html", "utf-8"))

        try:
            with self._create_connection() as server:
                server.sendmail(
                    self.config.username,
                    recipients,
                    msg.as_string()
                )
            return True
        except smtplib.SMTPException as e:
            print(f"이메일 발송 실패: {e}")
            return False


# 사용 예시
config = EmailConfig(
    smtp_host="smtp.gmail.com",
    smtp_port=587,
    username="your_email@gmail.com",
    password="your_app_password"
)

sender = EmailSender(config)
sender.send(
    to=["alice@example.com", "bob@example.com"],
    subject="주간 보고서",
    body="이번 주 업무 보고서를 첨부합니다.",
    cc=["manager@example.com"]
)
```

18.2 LLM 기반 이메일 내용 자동 생성

LLM을 활용한 이메일 작성의 장점

LLM(Large Language Model)을 이메일 작성에 연동하면 상황에 맞는 자연스러운 이메일을 자동으로 생성할 수 있습니다. 단순 템플릿 방식과 비교하면 다음과 같은 차이가 있습니다.

방식	장점	단점
템플릿 방식	일관성 높음, 비용 없음	유연성 부족, 다양한 상황 대응 어려움
LLM 방식	맥락 이해, 자연스러운 문체, 다국어 지원	API 비용 발생, 응답 시간 소요
하이브리드 방식	두 방식의 장점 결합	구현 복잡도 증가

기본 이메일 생성 시스템

```
from anthropic import Anthropic

client = Anthropic()

def generate_email(
    purpose: str,
    recipient: str,
    key_points: list[str],
    tone: str = "formal",
    language: str = "Korean"
) → dict:
    """LLM으로 이메일 내용 생성"""

    points_text = "\n".join(f"- {point}" for point in key_points)

    prompt = f"""다음 조건에 맞는 이메일을 작성해주세요.

발송 목적: {purpose}
수신자: {recipient}
어조: {tone} (formal=공식적, friendly=친근한, urgent=긴급한)
언어: {language}
핵심 내용:
{points_text}

다음 형식으로 반환해주세요:
제목: [이메일 제목]
---
[이메일 본문]"""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}]
    )

    # 제목과 본문 파싱
    full_text = response.content[0].text
    parts = full_text.split("---", 1)

    subject_line = parts[0].strip()
```

```
    subject = subject_line.replace("제목:", "").strip()
    body = parts[1].strip() if len(parts) > 1 else full_text

    return {"subject": subject, "body": body}


# 사용 예시
email_content = generate_email(
    purpose="프로젝트 진행 상황 보고",
    recipient="김팀장님",
    key_points=[
        "1단계 개발 완료 (예정 대비 3일 앞당김)",
        "2단계 착수 예정: 다음 주 월요일",
        "예산 사용률: 전체 예산의 45% 집행",
        "다음 주 목요일 중간 점검 회의 요청"
    ],
    tone="formal"
)

print("제목:", email_content["subject"])
print("본문:", email_content["body"])
```

다양한 이메일 유형 자동 생성

```
from anthropic import Anthropic
from enum import Enum

client = Anthropic()

class EmailType(Enum):
    MEETING_REQUEST = "회의 요청"
    FOLLOW_UP = "후속 조치"
    APOLOGY = "사과"
    INTRODUCTION = "자기소개"
    PROPOSAL = "제안서"
    REPORT = "보고서"


def generate_email_by_type(
    email_type: EmailType,
    context: dict
) → dict:
    """이메일 유형별 특화된 생성 함수"""

    type_instructions = {
        EmailType.MEETING_REQUEST: "회의 목적, 안건, 희망 일정을 명확히 포함하세요.",
        EmailType.FOLLOW_UP: "이전 논의 내용을 간략히 언급하고 후속 조치 사항을 명시하세요.",
        EmailType.APOLOGY: "진심 어린 사과와 함께 재발 방지 방안을 포함하세요.",
        EmailType.INTRODUCTION: "간결하고 임팩트 있게 본인을 소개하세요.",
        EmailType.PROPOSAL: "문제점, 해결책, 기대효과를 구조적으로 제시하세요.",
        EmailType.REPORT: "핵심 수치와 현황을 요약하고 다음 단계를 명시하세요.",
    }

    system_prompt = f"""당신은 전문적인 비즈니스 이메일 작성 전문가입니다.
이메일 유형: {email_type.value}
작성 지침: {type_instructions[email_type]}

항상 다음 형식으로 응답하세요:
제목: [제목 내용]
본문: [본문 내용]"""

    user_prompt = f"다음 정보로 이메일을 작성해주세요:\n{context}"

    response = client.messages.create(
```

```
        model="claude-opus-4-5",
        max_tokens=1024,
        system=system_prompt,
        messages=[{"role": "user", "content": user_prompt}]
    )

    text = response.content[0].text
    lines = text.split("\n")

    subject = ""
    body_lines = []
    is_body = False

    for line in lines:
        if line.startswith("제목:"):
            subject = line.replace("제목:", "").strip()
        elif line.startswith("본문:"):
            is_body = True
            rest = line.replace("본문:", "").strip()
            if rest:
                body_lines.append(rest)
        elif is_body:
            body_lines.append(line)

    return {
        "type": email_type.value,
        "subject": subject,
        "body": "\n".join(body_lines).strip()
    }


# 회의 요청 이메일 생성
meeting_email = generate_email_by_type(
    EmailType.MEETING_REQUEST,
    context={
        "수신자": "개발팀 전체",
        "회의 목적": "Q2 스프린트 계획 수립",
        "안건": ["목표 설정", "역할 분배", "일정 확정"],
        "희망 일정": "이번 주 금요일 오후 2시~4시",
        "장소": "3층 대회의실 또는 화상회의"
    }
)
```

```
print(f"[{meeting_email['type']}]")
print("제목:", meeting_email["subject"])
print("본문:", meeting_email["body"])
```

이메일 개선 및 교정

```
from anthropic import Anthropic

client = Anthropic()

def improve_email(
    original_email: str,
    improvement_goals: list[str]
) → str:
    """기존 이메일 내용 개선"""

    goals_text = "\n".join(f"- {goal}" for goal in improvement_goals)

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{
            "role": "user",
            "content": f"""다음 이메일을 개선해주세요.

[원본 이메일]
{original_email}

[개선 목표]
{goals_text}

개선된 이메일만 반환해주세요."""
        }]
    )

    return response.content[0].text

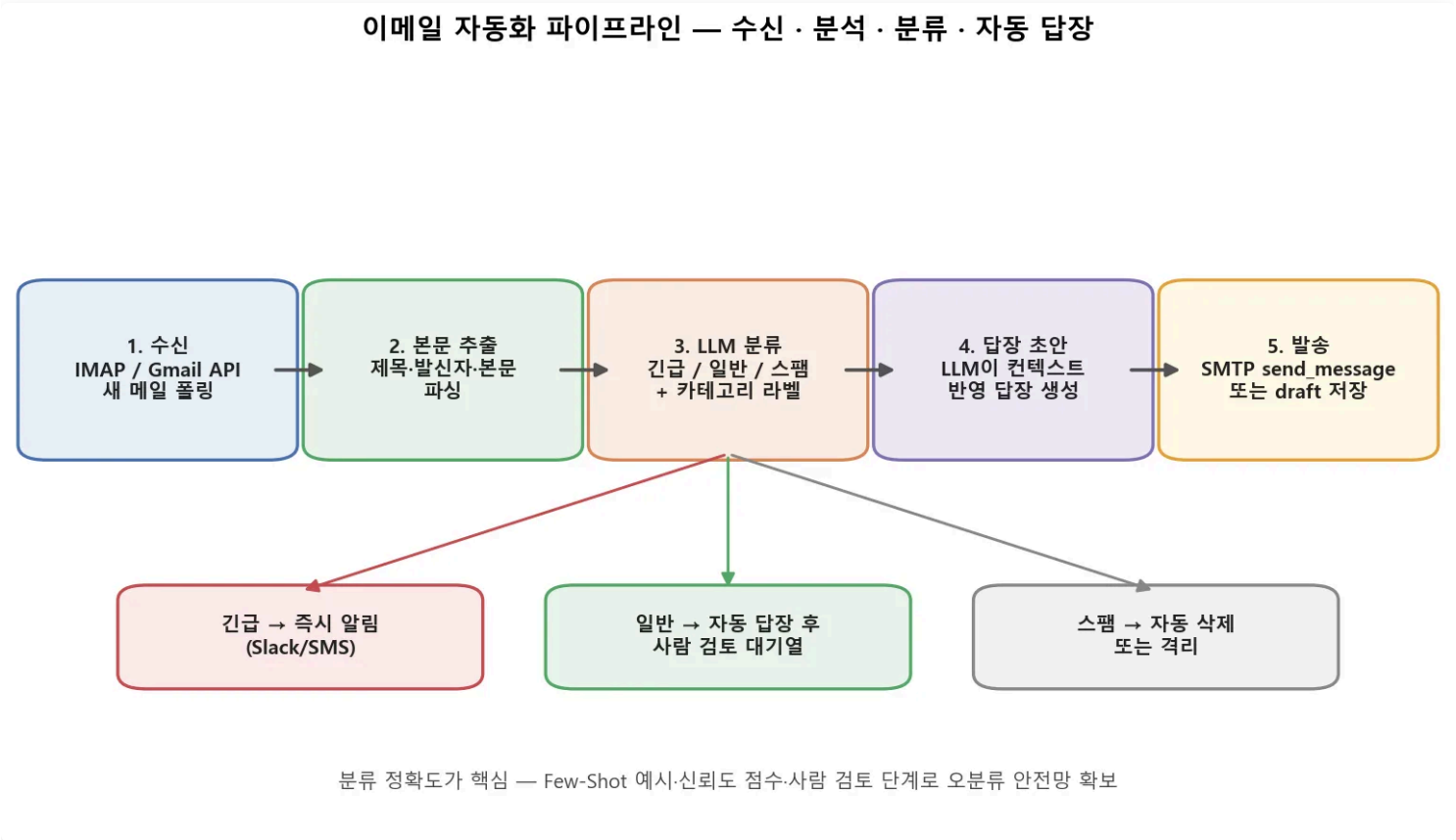

# 사용 예시
original = """안녕하세요
프로젝트 건으로 연락드립니다
일정이 좀 늦어질 것 같아요
죄송합니다
나중에 다시 연락드리겠습니다"""

improved = improve_email(
```

```
    original_email=original,
    improvement_goals=[
        "더 전문적이고 공식적인 어조로 수정",
        "구체적인 일정 지연 사유 추가",
        "대안적 일정 제안 포함",
        "경어 표현 올바르게 사용"
    ]
)

print(improved)
```

18.3 수신 이메일 분석 및 자동 분류



IMAP을 이용한 이메일 수신

IMAP(Internet Message Access Protocol)을 사용하면 서버에서 이메일을 읽고 관리할 수 있습니다. 파이썬 표준 라이브러리의 **imaplib**을 사용합니다.

```
import imaplib
import email
from email.header import decode_header
from dataclasses import dataclass
from typing import Optional


@dataclass
class EmailMessage:
    """이메일 메시지 데이터 클래스"""
    uid: str
    subject: str
    sender: str
    date: str
    body: str
    is_read: bool


def decode_str(value: str) → str:
    """이메일 헤더 디코딩"""
    decoded_parts = decode_header(value)
    result = []
    for part, charset in decoded_parts:
        if isinstance(part, bytes):
            charset = charset or "utf-8"
            result.append(part.decode(charset, errors="replace"))
        else:
            result.append(part)
    return "".join(result)


def fetch_emails(
    imap_host: str,
    username: str,
    password: str,
    folder: str = "INBOX",
    limit: int = 10,
    unread_only: bool = False
) → list[EmailMessage]:
    """IMAP으로 이메일 수신"""

    messages = []
```

```
    with imaplib.IMAP4_SSL(imap_host) as mail:
        mail.login(username, password)
        mail.select(folder)

        # 검색 조건
        search_criteria = "UNSEEN" if unread_only else "ALL"
        _, message_ids = mail.search(None, search_criteria)

        uid_list = message_ids[0].split()
        # 최신 limit개만 가져오기
        uid_list = uid_list[-limit:]

        for uid in reversed(uid_list):
            _, msg_data = mail.fetch(uid, "(RFC822 FLAGS)")

            raw_email = msg_data[0][1]
            flags = msg_data[0][0].decode()
            is_read = "\\Seen" in flags

            msg = email.message_from_bytes(raw_email)

            subject = decode_str(msg.get("Subject", ""))
            sender = decode_str(msg.get("From", ""))
            date = msg.get("Date", "")

            # 본문 추출
            body = ""
            if msg.is_multipart():
                for part in msg.walk():
                    if part.get_content_type() == "text/plain":
                        charset = part.get_content_charset() or "utf-8"
                        body = part.get_payload(decode=True).decode(
                            charset, errors="replace"
                        )
                        break
            else:
                charset = msg.get_content_charset() or "utf-8"
                body = msg.get_payload(decode=True).decode(
                    charset, errors="replace"
                )

            messages.append(EmailMessage(
```

```
                uid=uid.decode(),
                subject=subject,
                sender=sender,
                date=date,
                body=body[:500],  # 본문 앞 500자만
                is_read=is_read
            ))

    return messages
```

LLM 기반 이메일 자동 분류

```
from anthropic import Anthropic
import json
from enum import Enum

client = Anthropic()


class EmailCategory(Enum):
    URGENT = "긴급"
    MEETING = "회의/일정"
    REPORT = "보고/결과"
    REQUEST = "요청/협조"
    INFO = "정보/공지"
    PERSONAL = "개인"
    SPAM = "스팸/광고"
    OTHER = "기타"


def classify_email(email_msg: EmailMessage) → dict:
    """LLM으로 이메일 분류 및 분석"""

    prompt = f"""다음 이메일을 분석하고 JSON 형식으로 분류 결과를 반환해주세요.

제목: {email_msg.subject}
발신자: {email_msg.sender}
본문 (일부): {email_msg.body[:300]}

다음 JSON 형식으로만 응답해주세요:
{{
    "category": "긴급|회의/일정|보고/결과|요청/협조|정보/공지|개인|스팸/광고|기타",
    "priority": "high|medium|low",
    "summary": "한 문장 요약",
    "action_required": true/false,
    "action_description": "필요한 조치 사항 (없으면 null)",
    "keywords": ["키워드1", "키워드2"]
}}"""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
```

```
        messages=[{"role": "user", "content": prompt}]
    )

    try:
        result = json.loads(response.content[0].text)
        result["uid"] = email_msg.uid
        result["subject"] = email_msg.subject
        result["sender"] = email_msg.sender
        return result
    except json.JSONDecodeError:
        return {
            "uid": email_msg.uid,
            "subject": email_msg.subject,
            "category": "기타",
            "priority": "low",
            "summary": "분류 실패",
            "action_required": False
        }


def classify_multiple_emails(
    emails: list[EmailMessage]
) → list[dict]:
    """여러 이메일 일괄 분류"""
    results = []
    for email_msg in emails:
        result = classify_email(email_msg)
        results.append(result)
        print(f"분류 완료: [{result['category']}] {result['subject']}")
    return results


def generate_inbox_summary(classified_emails: list[dict]) → str:
    """받은 편지함 요약 생성"""

    urgent_count = sum(
        1 for e in classified_emails if e.get("priority") == "high"
    )
    action_needed = [
        e for e in classified_emails if e.get("action_required")
    ]

    summary_data = {
```

```
        "총 이메일 수": len(classified_emails),
        "긴급 이메일": urgent_count,
        "조치 필요": len(action_needed),
        "분류별 현황": {}
    }

    for email_info in classified_emails:
        cat = email_info.get("category", "기타")
        summary_data["분류별 현황"][cat] = \
            summary_data["분류별 현황"].get(cat, 0) + 1

    prompt = f"""다음 받은 편지함 현황을 바탕으로 간결한 요약 보고를 작성해주세요.

현황: {json.dumps(summary_data, ensure_ascii=False, indent=2)}

조치가 필요한 이메일:
{chr(10).join([f'- [{e["priority"]}] {e["subject"]}: {e.get("action_description", "")}' for e
in action_needed])}

3~5문장으로 간결하게 작성해주세요."""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        messages=[{"role": "user", "content": prompt}]
    )

    return response.content[0].text
```

18.4 예약 발송 시스템 구현

APScheduler를 이용한 예약 발송

APScheduler(Advanced Python Scheduler)는 파이썬에서 작업 예약을 위한 강력한 라이브러리입니다.

```
pip install apscheduler
```

```python
from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.cron import CronTrigger
from apscheduler.triggers.date import DateTrigger
from datetime import datetime, timedelta
import pytz
from dataclasses import dataclass, field
from typing import Callable, Optional
import uuid


@dataclass
class ScheduledEmail:
    """예약 이메일 데이터 클래스"""
    job_id: str
    to: str
    subject: str
    body: str
    scheduled_time: datetime
    status: str = "pending"  # pending, sent, failed, cancelled


class EmailScheduler:
    """이메일 예약 발송 시스템"""

    def __init__(self, email_sender: "EmailSender"):
        self.sender = email_sender
        self.scheduler = BackgroundScheduler(
            timezone=pytz.timezone("Asia/Seoul")
        )
        self.scheduled_jobs: dict[str, ScheduledEmail] = {}
        self.scheduler.start()
        print("이메일 예약 발송 시스템 시작")

    def schedule_once(
        self,
        to: str,
        subject: str,
        body: str,
        send_at: datetime
    ) -> str:
        """특정 시각에 한 번 발송 예약"""
        job_id = str(uuid.uuid4())[:8]
```

```
        email_info = ScheduledEmail(
            job_id=job_id,
            to=to,
            subject=subject,
            body=body,
            scheduled_time=send_at
        )
        self.scheduled_jobs[job_id] = email_info

        self.scheduler.add_job(
            func=self._send_job,
            trigger=DateTrigger(run_date=send_at),
            args=[job_id],
            id=job_id
        )

        print(f"이메일 예약 완료 [ID: {job_id}]: {send_at.strftime('%Y-%m-%d %H:%M')}")
        return job_id

    def schedule_recurring(
        self,
        to: str,
        subject_template: str,
        body_generator: Callable[[], str],
        cron_expression: str
    ) → str:
        """반복 발송 예약 (cron 표현식 사용)"""
        job_id = str(uuid.uuid4())[:8]

        self.scheduler.add_job(
            func=self._send_recurring,
            trigger=CronTrigger.from_crontab(cron_expression),
            args=[to, subject_template, body_generator],
            id=job_id
        )

        print(f"반복 발송 예약 [ID: {job_id}]: {cron_expression}")
        return job_id

    def _send_job(self, job_id: str):
        """예약된 이메일 발송 실행"""
        if job_id not in self.scheduled_jobs:
```

```
            return

        job = self.scheduled_jobs[job_id]
        success = self.sender.send(
            to=job.to,
            subject=job.subject,
            body=job.body
        )
        job.status = "sent" if success else "failed"
        print(f"예약 이메일 발송 {'완료' if success else '실패'} [ID: {job_id}]")

    def _send_recurring(
        self,
        to: str,
        subject_template: str,
        body_generator: Callable[[], str]
    ):
        """반복 발송 실행"""
        now = datetime.now(pytz.timezone("Asia/Seoul"))
        subject = subject_template.format(
            date=now.strftime("%Y-%m-%d"),
            time=now.strftime("%H:%M")
        )
        body = body_generator()
        self.sender.send(to=to, subject=subject, body=body)

    def cancel(self, job_id: str) → bool:
        """예약 취소"""
        try:
            self.scheduler.remove_job(job_id)
            if job_id in self.scheduled_jobs:
                self.scheduled_jobs[job_id].status = "cancelled"
            print(f"예약 취소 완료 [ID: {job_id}]")
            return True
        except Exception as e:
            print(f"예약 취소 실패: {e}")
            return False

    def list_jobs(self) → list[dict]:
        """예약된 작업 목록 조회"""
        jobs = []
        for job in self.scheduler.get_jobs():
            info = {
```

```
                "id": job.id,
                "next_run": str(job.next_run_time)
            }
            if job.id in self.scheduled_jobs:
                email_info = self.scheduled_jobs[job.id]
                info.update({
                    "to": email_info.to,
                    "subject": email_info.subject,
                    "status": email_info.status
                })
            jobs.append(info)
        return jobs

    def shutdown(self):
        """스케줄러 종료"""
        self.scheduler.shutdown()
        print("이메일 예약 발송 시스템 종료")


# 사용 예시
# scheduler = EmailScheduler(email_sender=sender)

# 30분 후 발송 예약
# send_time = datetime.now(pytz.timezone("Asia/Seoul")) + timedelta(minutes=30)
# job_id = scheduler.schedule_once(
#     to="manager@example.com",
#     subject="오늘의 업무 보고",
#     body="오늘 처리한 업무 내역을 보고드립니다...",
#     send_at=send_time
# )

# 매주 월요일 오전 9시 반복 발송
# scheduler.schedule_recurring(
#     to="team@example.com",
#     subject_template="주간 업무 현황 ({date})",
#     body_generator=lambda: "이번 주 업무 계획입니다...",
#     cron_expression="0 9 * * MON"
# )
```

큐(Queue) 기반 안정적인 발송 시스템

대량 이메일 발송 시에는 **Queue**를 활용하여 안정적으로 처리합니다.

```
import queue
import threading
import time
from dataclasses import dataclass
from typing import Optional


@dataclass
class EmailTask:
    """이메일 작업 데이터 클래스"""
    to: str
    subject: str
    body: str
    priority: int = 5  # 1(높음) ~ 10(낮음)
    retry_count: int = 0
    max_retries: int = 3


class EmailQueue:
    """큐 기반 이메일 발송 시스템"""

    def __init__(self, email_sender: "EmailSender", workers: int = 2):
        self.sender = email_sender
        self.queue = queue.PriorityQueue()
        self.workers = workers
        self.running = False
        self._threads: list[threading.Thread] = []

    def enqueue(self, task: EmailTask):
        """이메일 작업을 큐에 추가"""
        # priority 낮을수록 먼저 처리 (PriorityQueue)
        self.queue.put((task.priority, time.time(), task))
        print(f"큐 추가: [{task.priority}순위] {task.subject}")

    def start(self):
        """워커 스레드 시작"""
        self.running = True
        for i in range(self.workers):
            t = threading.Thread(
                target=self._worker,
                name=f"EmailWorker-{i+1}",
                daemon=True
```

```
            )
            t.start()
            self._threads.append(t)
        print(f"{self.workers}개 워커 스레드 시작")

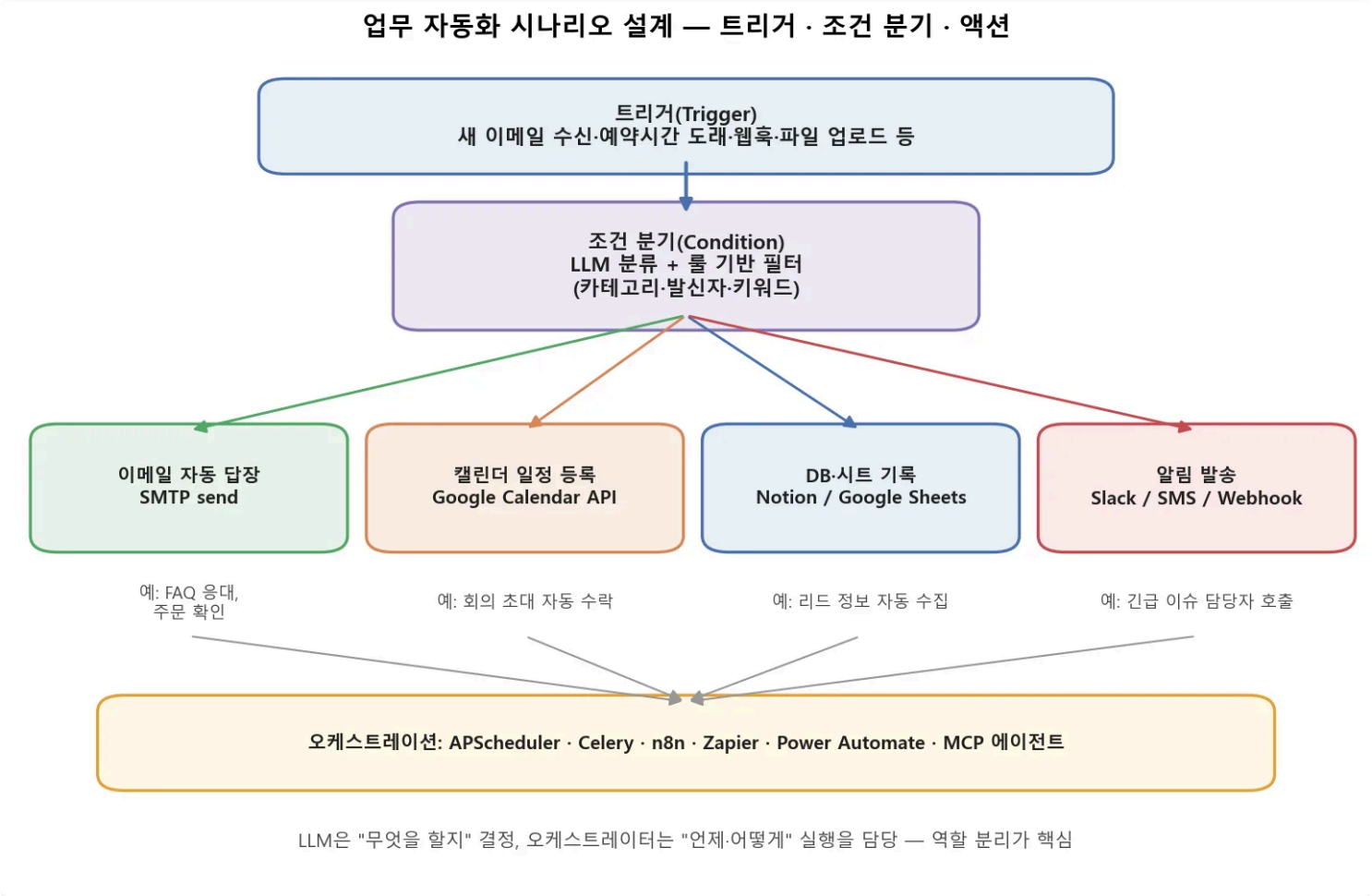
    def _worker(self):
        """워커 스레드: 큐에서 작업을 꺼내 처리"""
        while self.running:
            try:
                priority, _, task = self.queue.get(timeout=1)
                self._process_task(task)
                self.queue.task_done()
            except queue.Empty:
                continue

    def _process_task(self, task: EmailTask):
        """이메일 발송 처리 (재시도 포함)"""
        success = self.sender.send(
            to=task.to,
            subject=task.subject,
            body=task.body
        )

        if not success and task.retry_count < task.max_retries:
            task.retry_count += 1
            print(f"재시도 ({task.retry_count}/{task.max_retries}): {task.subject}")
            time.sleep(2 ** task.retry_count)  # 지수 백오프
            self.enqueue(task)

    def stop(self):
        """발송 시스템 종료 (큐 처리 완료 후)"""
        self.queue.join()  # 모든 작업 완료 대기
        self.running = False
        print("이메일 큐 시스템 종료")
```

18.5 업무 자동화 시나리오 설계



보고서 자동 작성 시스템

LLM을 활용하여 데이터를 분석하고 자동으로 보고서를 작성하는 시스템입니다.

```
from anthropic import Anthropic
from datetime import datetime, date
import json

client = Anthropic()


def generate_weekly_report(
    team_name: str,
    week_data: dict
) → dict:
    """주간 보고서 자동 생성"""

    data_str = json.dumps(week_data, ensure_ascii=False, indent=2)
    today = date.today().strftime("%Y년 %m월 %d일")

    prompt = f"""다음 데이터를 바탕으로 {team_name}팀의 주간 업무 보고서를 작성해주세요.
기준일: {today}

[주간 데이터]
{data_str}

보고서 구성:
1. 이번 주 성과 요약 (3~5줄)
2. 주요 완료 업무 (bullet point)
3. 진행 중인 업무 현황
4. 이슈 및 리스크
5. 다음 주 계획
6. 특이사항 및 건의사항

전문적이고 간결하게 작성해주세요."""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=2048,
        messages=[{"role": "user", "content": prompt}]
    )

    report_content = response.content[0].text

    return {
        "title": f"{team_name}팀 주간 보고서 ({today})",
```

```
        "content": report_content,
        "generated_at": datetime.now().isoformat()
    }


# 사용 예시
week_data = {
    "완료된 태스크": [
        {"이름": "API 성능 최적화", "담당자": "김개발", "소요시간": "3일"},
        {"이름": "사용자 인증 버그 수정", "담당자": "이디버그", "소요시간": "1일"},
        {"이름": "신규 기능 UI 개발", "담당자": "박프론트", "소요시간": "4일"}
    ],
    "진행 중인 태스크": [
        {"이름": "DB 마이그레이션", "진행률": "60%", "예상완료": "다음주 수요일"},
        {"이름": "테스트 자동화 구축", "진행률": "30%", "예상완료": "2주 후"}
    ],
    "KPI 달성률": {
        "배포 건수": "목표 5건 중 4건 완료 (80%)",
        "버그 해결률": "접수 12건 중 10건 해결 (83%)",
        "코드 커버리지": "72% (목표: 75%)"
    },
    "이슈": [
        "서버 응답 지연 현상 간헐적 발생 - 원인 분석 중",
        "코드 커버리지 목표 미달 - 다음 주 집중 개선 예정"
    ]
}

report = generate_weekly_report("개발", week_data)
print(report["title"])
print(report["content"])
```

일정 관리 자동화

```
from anthropic import Anthropic
from datetime import datetime, timedelta
import json

client = Anthropic()


def parse_schedule_request(natural_text: str) → dict:
    """자연어로 작성된 일정 요청을 파싱"""

    today = datetime.now().strftime("%Y-%m-%d %H:%M")

    prompt = f"""현재 날짜/시간: {today}

다음 자연어 일정 요청을 분석하여 JSON으로 변환해주세요:
"{natural_text}"

반환 형식:
{{
    "title": "일정 제목",
    "date": "YYYY-MM-DD",
    "time": "HH:MM",
    "duration_minutes": 60,
    "participants": ["참석자1", "참석자2"],
    "location": "장소 또는 null",
    "description": "상세 내용",
    "reminders": [30, 60]  // 몇 분 전 알림
}}"""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        messages=[{"role": "user", "content": prompt}]
    )

    try:
        return json.loads(response.content[0].text)
    except json.JSONDecodeError:
        return {"error": "파싱 실패", "raw": response.content[0].text}
```

```
def generate_meeting_invite(schedule: dict) → dict:
    """일정 정보로 회의 초대 이메일 생성"""

    prompt = f"""다음 일정 정보로 회의 초대 이메일을 작성해주세요.

일정 정보:
{json.dumps(schedule, ensure_ascii=False, indent=2)}

형식:
제목: [이메일 제목]
본문: [이메일 본문 - 일정 상세, 참석 확인 요청 포함]"""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        messages=[{"role": "user", "content": prompt}]
    )

    text = response.content[0].text
    parts = text.split("본문:", 1)

    subject_part = parts[0].replace("제목:", "").strip()
    body_part = parts[1].strip() if len(parts) > 1 else text

    return {"subject": subject_part, "body": body_part}


# 자연어 일정 파싱 예시
schedule = parse_schedule_request(
    "다음 주 화요일 오후 3시에 마케팅팀과 Q2 캠페인 전략 회의, "
    "2시간 예정, 2층 소회의실, 30분 전 알림"
)

invite = generate_meeting_invite(schedule)
print("회의 초대 이메일:")
print("제목:", invite["subject"])
print("본문:", invite["body"])
```

알림 시스템 구현

```
from anthropic import Anthropic
from dataclasses import dataclass
from enum import Enum
from typing import Callable
import threading
import time

client = Anthropic()


class AlertLevel(Enum):
    INFO = "정보"
    WARNING = "주의"
    CRITICAL = "긴급"


@dataclass
class AlertRule:
    """알림 규칙 데이터 클래스"""
    name: str
    condition: Callable[[], bool]
    level: AlertLevel
    recipients: list[str]
    message_template: str
    cooldown_minutes: int = 60  # 최소 알림 간격


class SmartAlertSystem:
    """LLM 기반 스마트 알림 시스템"""

    def __init__(self, email_sender: "EmailSender"):
        self.sender = email_sender
        self.rules: list[AlertRule] = []
        self.last_sent: dict[str, float] = {}

    def add_rule(self, rule: AlertRule):
        """알림 규칙 추가"""
        self.rules.append(rule)
        print(f"알림 규칙 추가: {rule.name} [{rule.level.value}]")
```

```
    def _should_send(self, rule_name: str, cooldown: int) → bool:
        """쿨다운 체크: 최소 발송 간격 확인"""
        last = self.last_sent.get(rule_name, 0)
        elapsed = time.time() - last
        return elapsed > cooldown * 60

    def generate_alert_message(
        self,
        rule: AlertRule,
        context: dict
    ) → str:
        """LLM으로 상황에 맞는 알림 메시지 생성"""

        prompt = f"""다음 알림 상황에 맞는 이메일 본문을 작성해주세요.

알림 유형: {rule.level.value}
규칙 이름: {rule.name}
템플릿: {rule.message_template}
상황 데이터: {context}

간결하고 명확하게, 즉시 조치해야 할 내용을 강조하여 작성해주세요."""

        response = client.messages.create(
            model="claude-opus-4-5",
            max_tokens=256,
            messages=[{"role": "user", "content": prompt}]
        )

        return response.content[0].text

    def check_and_notify(self, context: dict = None):
        """모든 규칙 점검 및 필요 시 알림 발송"""
        context = context or {}
        triggered = []

        for rule in self.rules:
            try:
                if rule.condition() and self._should_send(
                    rule.name, rule.cooldown_minutes
                ):
                    message = self.generate_alert_message(rule, context)
                    subject = f"[{rule.level.value}] {rule.name}"
```

```
                    for recipient in rule.recipients:
                        self.sender.send(
                            to=recipient,
                            subject=subject,
                            body=message
                        )

                    self.last_sent[rule.name] = time.time()
                    triggered.append(rule.name)
                    print(f"알림 발송: {rule.name} → {rule.recipients}")

            except Exception as e:
                print(f"알림 규칙 처리 오류 ({rule.name}): {e}")

        return triggered


# 사용 예시
# alert_system = SmartAlertSystem(email_sender=sender)
#
# alert_system.add_rule(AlertRule(
#     name="서버 CPU 과부하",
#     condition=lambda: get_cpu_usage() > 90,
#     level=AlertLevel.CRITICAL,
#     recipients=["ops@example.com", "cto@example.com"],
#     message_template="CPU 사용률이 {value}%로 임계치(90%)를 초과했습니다.",
#     cooldown_minutes=30
# ))
```

[실습] AI 이메일 자동화 시스템 구축

이번 실습에서는 지금까지 배운 내용을 통합하여 완전한 **AI 이메일 자동화 시스템**을 구축합니다.

시스템 아키텍처

```
[수신 이메일] → [IMAP 수신] → [LLM 분류/분석] → [자동 응답 or 전달]
[보고서 요청] → [LLM 보고서 생성] → [이메일 작성] → [예약/즉시 발송]
[알림 이벤트] → [조건 판단] → [LLM 메시지 생성] → [즉시 발송]
```

통합 시스템 구현

```
import os
import json
import time
import threading
from datetime import datetime, timedelta
from dataclasses import dataclass, field
from typing import Optional
import pytz
from anthropic import Anthropic

# 앞서 정의한 클래스들을 임포트한다고 가정
# from email_utils import EmailSender, EmailConfig, EmailMessage
# from email_classifier import classify_email
# from email_scheduler import EmailScheduler

client = Anthropic()


class AIEmailAutomationSystem:
    """AI 기반 이메일 자동화 통합 시스템"""

    def __init__(self, email_config: "EmailConfig"):
        self.config = email_config
        self.sender = EmailSender(email_config)
        self.scheduler = EmailScheduler(self.sender)
        self.running = False
        self.processed_uids: set[str] = set()

        # 자동 응답 규칙
        self.auto_reply_rules: list[dict] = []

    def add_auto_reply_rule(
        self,
        trigger_keywords: list[str],
        reply_template: str,
        category_filter: Optional[str] = None
    ):
        """자동 응답 규칙 추가"""
        self.auto_reply_rules.append({
            "keywords": [kw.lower() for kw in trigger_keywords],
```

```
            "template": reply_template,
            "category": category_filter
        })

    def _should_auto_reply(
        self,
        email_info: dict,
        classified: dict
    ) → Optional[str]:
        """자동 응답 여부 및 내용 결정"""
        subject_lower = email_info.get("subject", "").lower()
        body_lower = email_info.get("body", "").lower()

        for rule in self.auto_reply_rules:
            # 카테고리 필터 확인
            if rule["category"] and classified.get("category") ≠ rule["category"]:
                continue

            # 키워드 확인
            for keyword in rule["keywords"]:
                if keyword in subject_lower or keyword in body_lower:
                    return rule["template"]

        return None

    def _generate_auto_reply(
        self,
        original_email: "EmailMessage",
        template: str
    ) → str:
        """LLM으로 자동 응답 생성"""
        prompt = f"""다음 이메일에 대한 자동 응답을 작성해주세요.

원본 이메일:
제목: {original_email.subject}
발신자: {original_email.sender}
내용: {original_email.body[:200]}

응답 템플릿: {template}

자연스럽고 전문적인 응답 이메일 본문을 작성해주세요."""

        response = client.messages.create(
```

```
            model="claude-opus-4-5",
            max_tokens=512,
            messages=[{"role": "user", "content": prompt}]
        )

        return response.content[0].text

    def process_inbox(
        self,
        imap_host: str,
        check_interval: int = 300
    ):
        """받은 편지함 주기적 처리"""
        print(f"받은 편지함 모니터링 시작 (간격: {check_interval}초)")

        while self.running:
            try:
                # 새 이메일 수신
                new_emails = fetch_emails(
                    imap_host=imap_host,
                    username=self.config.username,
                    password=self.config.password,
                    unread_only=True,
                    limit=20
                )

                for email_msg in new_emails:
                    if email_msg.uid in self.processed_uids:
                        continue

                    # LLM 분류
                    classified = classify_email(email_msg)
                    print(
                        f"[{classified['category']}][{classified['priority']}] "
                        f"{email_msg.subject}"
                    )

                    # 긴급 이메일 즉시 알림
                    if classified.get("priority") == "high":
                        self.sender.send(
                            to=self.config.username,
                            subject=f"[긴급 알림] {email_msg.subject}",
                            body=f"긴급 이메일 수신:\n\n{classified['summary']}\n\n"
```

```
                            f"발신자: {email_msg.sender}"
                    )

                # 자동 응답 확인
                reply_template = self._should_auto_reply(
                    {"subject": email_msg.subject, "body": email_msg.body},
                    classified
                )

                if reply_template:
                    reply_body = self._generate_auto_reply(
                        email_msg, reply_template
                    )
                    # 발신자 주소 추출 (간단화)
                    sender_email = email_msg.sender.split("<")[-1].rstrip(">")
                    self.sender.send(
                        to=sender_email,
                        subject=f"Re: {email_msg.subject}",
                        body=reply_body
                    )
                    print(f"자동 응답 발송: {sender_email}")

                self.processed_uids.add(email_msg.uid)

            except Exception as e:
                print(f"받은 편지함 처리 오류: {e}")

            time.sleep(check_interval)

    def schedule_daily_report(
        self,
        report_recipient: str,
        report_generator: callable,
        send_hour: int = 9
    ):
        """매일 정해진 시각에 보고서 발송 예약"""
        def generate_and_send():
            report = report_generator()
            self.sender.send(
                to=report_recipient,
                subject=report["title"],
                body=report["content"]
            )
```

```
        # 매일 지정 시각에 실행 (APScheduler cron)
        self.scheduler.schedule_recurring(
            to=report_recipient,
            subject_template="일일 보고서 ({date})",
            body_generator=lambda: report_generator()["content"],
            cron_expression=f"0 {send_hour} * * *"
        )
        print(f"일일 보고서 예약: 매일 {send_hour}시")

    def start(self, imap_host: str):
        """시스템 시작"""
        self.running = True
        inbox_thread = threading.Thread(
            target=self.process_inbox,
            args=(imap_host,),
            daemon=True
        )
        inbox_thread.start()
        print("AI 이메일 자동화 시스템 가동 완료")

    def stop(self):
        """시스템 종료"""
        self.running = False
        self.scheduler.shutdown()
        print("AI 이메일 자동화 시스템 종료")


# ═══ 시스템 실행 예시 =====
def main():
    # 이메일 설정
    config = EmailConfig(
        smtp_host="smtp.gmail.com",
        smtp_port=587,
        username=os.environ.get("EMAIL_USER", "your@gmail.com"),
        password=os.environ.get("EMAIL_PASS", "your_app_password")
    )

    # 시스템 초기화
    system = AIEmailAutomationSystem(config)

    # 자동 응답 규칙 설정
    system.add_auto_reply_rule(
```

```
        trigger_keywords=["휴가", "부재중", "자리 비움"],
        reply_template="현재 부재중입니다. 긴급한 경우 연락처를 남겨주시면 복귀 후 연락드리겠습니다.",
    )
    system.add_auto_reply_rule(
        trigger_keywords=["견적", "가격 문의", "비용"],
        reply_template="견적 문의 감사합니다. 영업일 기준 1~2일 내에 상세 견적서를 보내드리겠습니다.",
        category_filter="요청/협조"
    )

    # 일일 보고서 예약
    system.schedule_daily_report(
        report_recipient="manager@example.com",
        report_generator=lambda: generate_weekly_report(
            "개발",
            {"완료": [], "진행중": [], "이슈": []}
        ),
        send_hour=9
    )

    # 시스템 시작
    system.start(imap_host="imap.gmail.com")

    try:
        # 메인 루프 유지
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        system.stop()


if __name__ == "__main__":
    main()
```

환경 변수 설정

보안을 위해 이메일 계정 정보는 반드시 환경 변수로 관리합니다.

```
# .env 파일
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_gmail_app_password
ANTHROPIC_API_KEY=your_anthropic_api_key
```

```
from dotenv import load_dotenv
import os

load_dotenv()

config = EmailConfig(
    smtp_host="smtp.gmail.com",
    smtp_port=587,
    username=os.environ["EMAIL_USER"],
    password=os.environ["EMAIL_PASS"]
)
```

시스템 테스트

```
import unittest
from unittest.mock import MagicMock, patch


class TestEmailAutomation(unittest.TestCase):
    """이메일 자동화 시스템 단위 테스트"""

    def setUp(self):
        self.mock_sender = MagicMock()
        self.mock_sender.send.return_value = True

    def test_email_generation(self):
        """이메일 생성 테스트"""
        result = generate_email(
            purpose="테스트 이메일",
            recipient="테스트 수신자",
            key_points=["테스트 항목 1", "테스트 항목 2"]
        )
        self.assertIn("subject", result)
        self.assertIn("body", result)
        self.assertGreater(len(result["body"]), 0)

    @patch("smtplib.SMTP")
    def test_email_send(self, mock_smtp):
        """이메일 발송 테스트 (Mock)"""
        mock_server = MagicMock()
        mock_smtp.return_value.__enter__ = MagicMock(return_value=mock_server)
        mock_smtp.return_value.__exit__ = MagicMock(return_value=False)

        config = EmailConfig(
            smtp_host="smtp.test.com",
            smtp_port=587,
            username="test@test.com",
            password="testpass"
        )
        sender = EmailSender(config)
        result = sender.send(
            to="recipient@test.com",
            subject="테스트",
            body="테스트 본문"
```

```
        )
        self.assertTrue(result)

    def test_email_classification(self):
        """이메일 분류 테스트"""
        test_email = EmailMessage(
            uid="001",
            subject="긴급: 서버 장애 발생",
            sender="ops@example.com",
            date="2026-04-01",
            body="현재 서버 응답이 없습니다. 즉시 확인이 필요합니다.",
            is_read=False
        )
        result = classify_email(test_email)
        self.assertIn("category", result)
        self.assertIn("priority", result)
        self.assertEqual(result["priority"], "high")


if __name__ == "__main__":
    unittest.main()
```

정리

이번 챕터에서는 파이썬과 LLM을 결합한 이메일 및 업무 자동화 시스템을 구축하는 방법을 학습했습니다.

주제	핵심 내용
SMTP/smtplib	표준 라이브러리로 이메일 발송, HTML·첨부 파일 지원
LLM 이메일 생성	상황별 맞춤 이메일 자동 작성, 교정 및 개선
IMAP 수신 분류	받은 편지함 자동 분석, 우선순위·카테고리 분류
APScheduler	예약 발송, 반복 발송, 큐 기반 안정적 처리
업무 자동화	보고서 자동 작성, 일정 관리, 스마트 알림 시스템

다음 챕터(Chapter 19)에서는 Part 7을 시작하며 데이터베이스와 SQL 기초를 다룹니다. LLM을 활용하여 자연어로 데이터베이스를 조작하고 쿼리를 자동 생성하는 방법을 학습합니다.